

Sinopsis

- 16.1 Introducción
- 16.2 Definiciones de estructuras
- 16.3 Cómo tener acceso a miembros de estructuras
- 16.4 Cómo poner en práctica mediante un struct un tipo Time definido por usuario
- 16.5 Cómo poner en práctica un tipo de dato abstracto Time con una clase
- 16.6 Alcance de clase y acceso a miembros de clase
- 16.7 Cómo separar la interfaz de la puesta en práctica
- 16.8 Cómo controlar el acceso a miembros
- 16.9 Funciones de acceso y funciones de utilidad
- 16.10 Cómo inicializar objetos de clase: constructores
- 16.11 Cómo utilizar argumentos por omisión con los constructores
- 16.12 Cómo utilizar destructores
- 16.13 Cuándo son llamados los destructores y los constructores
- 16.14 Cómo utilizar miembros de datos y funciones miembro
- 16.15 Una trampa sutil: cómo regresar una referencia a un miembro de datos privado
- 16.16 Asignación por omisión en copia a nivel de miembro
- 16.17 Reutilización del software

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Observación de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

16.1 Introducción

Ahora empezamos nuestra introducción a la orientación a objetos. Veremos que la orientación a objetos es una forma natural de pensar en relación con el mundo y de escribir programas de computación. ¿Por qué, entonces, desde la primera página de este libro no empezamos con la orientación a objetos? ¿Por qué estamos posponiendo hasta el capítulo 16 la programación orientada a objetos en C++? La respuesta es que los objetos que construiremos estarán compuestos en parte por piezas de programas estructurados, por lo que necesitábamos establecer primero una base de programación estructurada.

En los primeros capítulos, nos concentramos en la metodología “convencional” de la programación estructurada. En este capítulo, presentamos conceptos básicos (es decir, “objeto pensamiento”) y terminología (es decir “objeto lenguaje”) de la programación orientada a objetos. En capítulos subsecuentes, atacaremos temas más sustanciales y problemas retadores utilizando las técnicas del *diseño orientado a objetos* (OOD, por *object-oriented design*), analizamos enunciados de problemas típicos que requieren la construcción de sistemas, determinamos qué

objetos necesitamos para poner en práctica los sistemas, determinamos qué atributos tendrán que tener los objetos, determinamos qué comportamientos necesitarán mostrar dichos objetos y especificamos cómo deberán interactuar los objetos uno con el otro para cumplir con las metas generales del sistema.

Empezamos presentando parte de la terminología clave de la orientación a objetos. Mire a su alrededor en el mundo real. Por todas partes donde mire los verá *¡objetos!* Personas, animales, plantas, automóviles, aviones, edificios, cortadoras de pastos, computadoras y demás. Los seres humanos pensamos en términos de objetos. Tenemos la capacidad maravillosa de la *abstracción*, que nos permite ver una imagen en pantalla como personas, aviones, árboles y montañas, en vez de puntos individuales de color. Podemos, si así lo deseamos, pensar en términos de playas, en vez de en granos de arena, bosques en lugar de árboles, y casas en lugar de tabiques.

Pudiéramos estar inclinados a dividir los objetos en dos categorías: objetos animados y objetos inanimados. De alguna forma, los objetos animados están “vivos”. Se mueven y hacen cosas. Los objetos inanimados, como las toallas, no parecen hacer mucho. Simplemente “están ahí”. Todos estos objetos, sin embargo, sí tienen algunas cosas en común. Todos tienen *atributos*, como tamaño, forma, color, peso y demás. Todos ellos exhiben varios *comportamientos*, por ejemplo, un balón rueda, rebota, se infla y se desinfla; un bebé llora, duerme, gatea, camina y parpadea; un automóvil se puede acelerar, frenar, girar, etcétera.

Los seres humanos aprenden lo relacionado con los objetos estudiando sus atributos y observando su comportamiento. Objetos diferentes pueden tener muchos atributos iguales y mostrar comportamientos similares. Se pueden hacer comparaciones, por ejemplo, entre bebés y adultos, entre seres humanos y chimpancés. Automóviles, camiones, pequeños carros rojos y patines tienen mucho en común.

La *programación orientada a objetos* (OOP) hace modelos de los objetos del mundo real mediante sus contrapartes en software. Aprovecha las relaciones de clase, donde objetos de una cierta clase, como la clase de vehículos, tienen las mismas características. Aprovecha las relaciones de *herencia*, e inclusive las relaciones de *herencia múltiple*, donde clases recién creadas de objetos se derivan heredando características de clases existentes, pero poseyendo características únicas, propias de ellos mismos. Los bebés tienen muchas características de sus padres, pero ocasionalmente padres de baja estatura tienen hijos altos.

La programación orientada a objetos nos proporciona una forma más natural e intuitiva de observar el proceso de programación, es decir *haciendo modelos* de objetos del mundo real, de sus atributos y de sus comportamientos. OOP también hace modelos de la comunicación entre los objetos. De la misma forma que las personas se envían *mensajes* uno al otro (por ejemplo, un sargento al comando de tropas para ponerlas en posición de firmes), los objetos también se comunican mediante mensajes.

OOP *encapsula* datos (atributos) y funciones (comportamiento) en paquetes llamados *objetos*; los datos y las funciones de un objeto están muy unidos. Los objetos tienen la propiedad de *ocultar la información*. Esto significa que aunque los objetos puedan saber cómo comunicarse unos con otros mediante *interfaces* bien definidas, a los objetos por lo regular no se les está permitido saber cómo funcionan otros objetos los detalles de puesta en práctica quedan ocultos dentro de los objetos mismos. Seguramente es posible conducir un automóvil eficaz sin saber los detalles de cómo es el funcionamiento interno de motores, transmisiones y sistemas de escape. Veremos por qué este ocultamiento de la información es tan crucial para una buena ingeniería de software.

En C y en otros *lenguajes de programación procedurales*, la programación tiende a ser *orientada a la acción*, en tanto que en la programación C++ tiende a ser *orientada al objeto*. En

C, la unidad de programación es la *función*. En C++, la unidad de programación es la *clase* a partir de la cual eventualmente los objetos son *producidos* (es decir son creados).

Los programadores de C se concentran en escribir funciones. Grupos de acciones que ejecutan alguna tarea común se agrupan en funciones, y a su vez las funciones se agrupan para formar programas. Es cierto que en C los datos son importantes, pero la óptica es que los datos existen de forma primordial para soportar las acciones que las funciones ejecutan. Los *verbos* en una especificación de sistema ayudan al programador de C a determinar el conjunto de funciones que juntas funcionarán para poner en práctica el sistema.

Los programadores de C++ se concentran en crear sus propios *tipos definidos por el usuario* conocidos como *clases*. Cada clase contiene datos junto con un conjunto de funciones que manipula dichos datos. Los componentes de datos de una clase se llaman *miembros de datos*. Los componentes de función de una clase se llaman *funciones miembros*. Al igual que un ejemplo de un tipo incorporado como `int` se conoce como una *variable*, un ejemplo de un tipo definido por usuario (es decir, una clase) se conoce como un *objeto*. El foco de atención en C++ está sobre los objetos, en vez de sobre las funciones. Los *nombres* en una especificación de sistema ayudan al programador de C++ a determinar el conjunto de clases, a partir de las cuales serán creados los objetos que funcionarán conjuntamente para poner en práctica el sistema.

Las clases en C++ son un desarrollo o evolución natural de la idea de `struct` de C. Antes de continuar con los detalles del desarrollo de las clases en C++, repasaremos las estructuras y construiremos un tipo definido por el usuario basado en una estructura. La debilidad que expondremos mediante este enfoque, nos ayudará a racionalizar y motivar el concepto de clase.

16.2 Definiciones de estructura

Considere la siguiente definición de estructura:

```
struct Time {
    int hour;      // 0-23
    int minute;    // 0-59
    int second;    // 0-59
};
```

Demos un repaso a la terminología de las estructuras (vea los capítulos 10 y 12). La palabra reservada `struct` presenta la definición de estructura. El identificador `Time` es la *etiqueta de estructura*. La etiqueta de estructura le da nombre a la definición de la estructura, y se utiliza para declarar variables del tipo estructura. En este ejemplo, el nombre del tipo estructura es `Time` (a diferencia del nombre de tipo más largo `struct Time`, que se requiere en C). Los nombres declarados en las llaves de la definición de estructura, son los *miembros* de la estructura. Los miembros de una misma estructura deben tener nombres únicos, pero dos estructuras distintas pueden contener sin conflicto miembros con un mismo nombre. Cada definición de estructura debe terminar con un punto y coma. Como veremos pronto, la explicación anterior es también válida para las clases.

La definición de `Time` contiene tres miembros del tipo `int`: `hour`, `minute`, y `second`. Los miembros de estructura pueden ser de cualquier tipo. Una estructura no puede, sin embargo, contener una ocurrencia de sí misma. Por ejemplo, en la definición de estructura correspondiente a `Time` no puede ser declarado un miembro del tipo `Time`. Sin embargo, podría ser incluido un apuntador a una estructura `Time`. Una estructura que contenga un miembro, que es un apuntador al mismo tipo de estructura, se conoce como una *estructura autorreferenciada*. Las estructuras autorreferenciadas son útiles para formar estructuras de datos enlazados (vea el capítulo 12).

La definición anterior de estructura no reserva ningún espacio en memoria; más bien, la definición crea un nuevo tipo de datos, que es utilizado para declarar variables. Las variables de estructura se declaran, al igual que las variables de otros tipos. La declaración

```
Time timeObject, timeArray[10], *timePtr;
```

declara `TimeObject` como una variable del tipo `Time`, `TimeArray`, como un arreglo de 10 elementos del tipo `Time`, y `TimePtr` como un apuntador a un objeto `Time`.

16.3 Cómo tener acceso a miembros de estructuras

Se tiene acceso a miembros de una estructura (o de una clase) utilizando los *operadores de acceso a miembros* el *operador punto* (`.`) y el *operador flecha* (`->`). El operador punto da acceso a un miembro de estructura (o de clase) vía el nombre de variable del objeto o vía una referencia al objeto. Por ejemplo, para imprimir el miembro `hour` de la estructura `timeObject`, utilice el enunciado

```
cout << timeObject.hour;
```

El operador de flecha que consiste de un signo (`-`) y de un signo mayor que (`>`) sin espacios intermedios da acceso a un miembro de estructura (o de clase) vía un apuntador al objeto. Suponga que el apuntador `timePtr` ha sido declarado para señalar a un objeto `Time`, y que la dirección de la estructura `timeObject` ha sido asignada a `timePtr`. Para imprimir el miembro `hour` de la estructura `timeObject` mediante el apuntador `timePtr`, utilice el enunciado

```
cout << timePtr->hour;
```

La expresión `timePtr->hour` es equivalente a `(*timePtr).hour`, que desreferencia el apuntador y da acceso al miembro `hour` mediante el uso del operador de punto. Aquí se necesitan los paréntesis, porque el operador de punto (`.`) tiene una precedencia más alta que el operador de desreferenciación de apuntadores (`*`). El operador de flecha y el operador de miembro de estructura, junto con los paréntesis y los corchetes (`[]`) tienen la precedencia de operadores más alta (de todos los operadores hasta ahora analizados) y se asocian de izquierda a derecha.

16.4 Cómo poner en práctica mediante un struct un tipo Time definido por el usuario

En la figura 16.1 se crea un tipo de estructura definida por el usuario `Time` con tres miembros enteros: `hour`, `minute` y `second`. El programa define una estructura `Time` llamada `dinnerTime`, y utiliza el operador de punto para inicializar los miembros de la estructura con los valores 18 para `hour`, 30 para `minute` y 0 para `second`. A continuación, el programa imprime la hora en formato militar y en formato estándar. Note que las funciones de impresión reciben referencias a las estructuras constantes `Time`. Esto hace que las estructuras `Time` sean pasadas por referencia a las funciones de impresión eliminando por lo tanto, la sobrecarga de copia asociada al pasar por valor estructuras a las funciones y también evita que las funciones de impresión modifiquen la estructura `Time`. En el capítulo 17 analizaremos los objetos `const` y las funciones miembro `const`.

Sugerencia de rendimiento 16.1

Las estructuras por lo regular pasan en llamada por valor. Para evitar la sobrecarga de copiar una estructura, pase la estructura en llamada por referencia.

```

// FIG16_1.CPP
// Create a structure, set its members, and print it.
#include <iostream.h>

struct Time {      // structure definition
    int hour;      // 0-23
    int minute;    // 0-59
    int second;    // 0-59
};

void printMilitary(const Time &); // prototype
void printStandard(const Time &); // prototype

main()
{
    Time dinnerTime;    // variable of new type Time

    // set members to valid values
    dinnerTime.hour = 18;
    dinnerTime.minute = 30;
    dinnerTime.second = 0;

    cout << "Dinner will be held at ";
    printMilitary(dinnerTime);
    cout << " military time,\nwhich is ";
    printStandard(dinnerTime);
    cout << " standard time." << endl;

    // set members to invalid values
    dinnerTime.hour = 29;
    dinnerTime.minute = 73;
    dinnerTime.second = 103;

    cout << "\nTime with invalid values: ";
    printMilitary(dinnerTime);
    cout << endl;
    return 0;
}

// Print the time in military format
void printMilitary(const Time &t)
{
    cout << (t.hour < 10 ? "0" : "") << t.hour << ":"
        << (t.minute < 10 ? "0" : "") << t.minute << ":"
        << (t.second < 10 ? "0" : "") << t.second;
}

// Print the time in standard format
void printStandard(const Time &t)
{
    cout << ((t.hour == 0 || t.hour == 12) ? 12 : t.hour % 12)
        << ":" << (t.minute < 10 ? "0" : "") << t.minute
        << ":" << (t.second < 10 ? "0" : "") << t.second
        << (t.hour < 12 ? " AM" : " PM");
}

```

Fig. 16.1 Cómo crear una estructura, definir sus miembros e imprimirla (parte 1 de 2).

Dinner will be held at 18:30:00 military time,
which is 6:30:00 PM standard time.

Time with invalid values: 29:73:103

Fig. 16.1 Cómo crear una estructura, definir sus miembros e imprimirla (parte 2 de 2).

Sugerencia de rendimiento 16.2

A fin de evitar la sobrecarga de la llamada por valor y aún así obtener el beneficio de que la información original del llamador quede protegida contra modificaciones, pase argumentos de tamaño extenso como referencias `const`.

Existen inconvenientes para la creación de nuevos tipos de datos utilizando estructuras de esta forma. Dado que la inicialización no es específicamente requerida, es posible tener datos sin inicializar, con los problemas consiguientes. Aún si los datos están inicializados, pudieran no estar de forma correcta inicializados. A los miembros de una estructura se les pueden asignar valores inválidos (como lo hicimos en la figura 16.1) porque el programa tiene acceso directo a los datos. El programa asignó valores incorrectos a los tres miembros del objeto `time dinnerTime`. Si es modificada la puesta en marcha del `struct`, deberán de ser modificados todos los programas que utilizan el `struct`. Esto es debido a que el programador está manipulando de manera directa el tipo de datos. No existe "interfaz" con el programa para asegurarse que el programador utiliza correctamente el tipo de datos, y para asegurar que los datos se conservan en un estado consistente.

Observación de ingeniería de software 16.1

Es importante escribir programas que sean comprensibles y fáciles de mantener. Las modificaciones son la regla más que la excepción. Los programadores deberán prever que su código será modificado. Como veremos, las clases facilitan la capacidad de modificación de los programas.

Existen otros problemas asociados con las estructuras de tipo C. No pueden ser impresas como unidad, más bien sus miembros deben ser impresos, y se les debe dar formato uno por uno. Se podría escribir una función para imprimir los miembros de una estructura en algún formato apropiado. En el capítulo 18, "Homonimia de operadores", se ilustra cómo hacer la homonimia del operador `<<` para habilitar los objetos de un tipo de estructura o de clase para que sean impresos con facilidad. Las estructuras no pueden ser comparadas en su totalidad; deben ser comparadas miembro por miembro. En el capítulo 18 también se ilustra cómo hacer la homonimia de los operadores de igualdad y relacionales, a fin de comparar objetos de tipos de estructura y de clase.

En la sección siguiente se vuelve a poner en práctica nuestra estructura `Time` como una clase y se demuestran algunas de las ventajas de la creación de tipos de datos abstractos con las clases. Veremos que en C++ las clases y las estructuras pueden ser utilizadas prácticamente en forma idéntica. La diferencia entre ambas es la accesibilidad por omisión asociada con los miembros de cada una. Esto será explicado en breve.

16.5 Cómo poner en práctica un tipo de dato abstracto `Time` con una clase

Las clases permiten que el programador modele objetos que tienen *atributos* (representados como *miembros de datos*) y *comportamientos* u *operaciones* (representados como *funciones*).

miembro). Los tipos contienen miembros de datos y funciones miembro y en C++ son por lo regular definidos mediante la palabra reservada **class**.

Las funciones miembro en otros lenguajes de programación orientados a objetos a veces se llaman *métodos*, y son invocados en respuesta a *mensajes* enviados a un objeto. Un mensaje corresponde a una llamada de función miembro.

Una vez que se haya definido una clase, el nombre de la clase puede ser utilizado para declarar objetos de dicha clase. En la figura 16.2 se muestra una definición simple para la clase **Time**.

Nuestra definición de la clase **Time** empieza con la palabra reservada **class**. El *cuerpo* de la definición de clase se delimita mediante llaves izquierdas y derechas (**{** y **}**). La definición de clase termina con un punto y coma. Nuestra definición de clase **Time** y nuestra definición de estructura **Time** cada una de ellas contiene tres miembros enteros **hour**, **minute** y **second**.

Error común de programación 16.1

Olvidar el punto y coma al final de una definición de clase.

Las partes restantes de la definición de clase son nuevas. La etiqueta **public:** y **private:** se conocen como especificadores de acceso de miembro. Cualquier miembro de datos o función miembro declarado después del especificador de acceso de miembro **public:** (y antes del siguiente especificador de acceso de miembro) es accesible, siempre que el programa tenga acceso a un objeto de la clase **Time**. Cualquier miembro de datos o función miembro declarada después del especificador de acceso de miembro **private:** (y hasta el siguiente especificador de acceso de miembro) sólo es accesible a las funciones miembros de la clase. Los especificadores de acceso de miembro terminan siempre con dos puntos (**:**) y pueden aparecer varias veces en una definición de clase.

Práctica sana de programación 16.1

Para mayor claridad y legibilidad, utilice cada especificador de acceso de miembro sólo una vez en una definición de clase. Coloque primero los miembros públicos, donde sean fácilmente localizables.

La definición de clase contiene prototipos para las cuatro siguientes funciones miembros después del especificador de acceso de miembros **public:** **Time**, **setTime**, **printMilitary** y **printStandard**. Estas son las *funciones miembro públicas*, o *servicios públicos* o *interfaz de la clase*. Estas funciones serán usadas por los clientes (usuarios) de la clase para manipular los datos de la clase.

```
class Time {
public:
    Time();
    void setTime(int, int, int);
    void printMilitary();
    void printStandard();
private:
    int hour;      // 0 - 23
    int minute;    // 0 - 59
    int second;    // 0 - 59
};
```

Fig. 16.2 Definición simple de la **class Time**.

Una función miembro con el mismo nombre que la clase se llama una función *constructor* de dicha clase. Un constructor es una función miembro especial, que inicializa los miembros de datos de un objeto de clase. Cuando se crea un objeto de una clase se llama a la función constructor de dicha clase.

Los tres miembros enteros aparecen después del especificador de acceso de miembro **private:**. Esto indica que estos miembros de datos de la clase son accesibles sólo a las funciones miembro y como veremos en el siguiente capítulo, a los amigos de la clase. Entonces, los miembros de datos solamente son accesibles por las cuatro funciones cuyos prototipos aparecen en la definición de clase (o por amigos de la clase). Por lo regular los miembros de datos aparecen listados en la porción **private:** de una clase y normalmente las funciones miembro aparecen listados en la porción **public:**. Como veremos más adelante es posible que existan funciones miembro **private:** y datos **public:**.

Una vez definida la clase, puede ser utilizada como un tipo en declaraciones como sigue:

```
Time sunset,                // object of type Time
    arrayOfTimes[5],        // array of Time objects
    *pointerToTime,         // pointer to a Time object
    &dinnerTime = sunset;    // reference to a Time object
```

El nombre de clase se convierte en un nuevo especificador de tipo. Pueden existir muchos objetos de una clase, igual que pueden existir muchas variables del tipo **int**. El programador puede crear nuevos tipos de clase según requiera. Esta es una de las muchas razones por las que C++ es un *lenguaje extensible*.

En la figura 16.3 se utiliza la clase **Time**. El programa produce un objeto de la clase **Time** llamado **t**. Cuando se produce el objeto, en forma automática es llamado el constructor **Time** e inicializa a 0 cada miembro de dato privado. La hora es entonces impresa en formatos militar y estándar, a fin de confirmar que los miembros han sido inicializados de forma correcta. A continuación, se ajusta la hora utilizando la función miembro **setTime** y se vuelve a imprimir la hora en ambos formatos. A continuación la función miembro **setTime** intenta establecer los miembros de datos a valores inválidos, y otra vez se imprime la hora en ambos formatos.

```
// FIG16_3.CPP
// Time class.
#include <iostream.h>

// Time abstract data type (ADT) definition
class Time {
public:
    Time();                // default constructor
    void setTime(int, int, int); // set hour, minute and second
    void printMilitary();    // print military time format
    void printStandard();    // print standard time format

private:
    int hour;              // 0 - 23
    int minute;            // 0 - 59
    int second;            // 0 - 59
};
```

Fig. 16.3 Puesta en práctica de tipos de datos abstractos **Time** como una clase (parte 1 de 3).

```

// Time constructor initializes each data member to zero.
// Ensures all Time objects start in a consistent state.
Time::Time() { hour = minute = second = 0; }

// Set a new Time value using military time. Perform validity
// checks on the data values. Set invalid values to zero.
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Print Time in military format
void Time::printMilitary()
{
    cout << (hour < 10 ? "0" : "") << hour << ":"
        << (minute < 10 ? "0" : "") << minute << ":"
        << (second < 10 ? "0" : "") << second;
}

// Print time in standard format
void Time::printStandard()
{
    cout << ((hour == 0 || hour == 12) ? 12 : hour % 12)
        << ":" << (minute < 10 ? "0" : "") << minute
        << ":" << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}

// Driver to test simple class Time
main()
{
    Time t; // instantiate object t of class Time

    cout << "The initial military time is ";
    t.printMilitary();
    cout << "\nThe initial standard time is ";
    t.printStandard();

    t.setTime(13, 27, 6);
    cout << "\n\nMilitary time after setTime is ";
    t.printMilitary();
    cout << "\nStandard time after setTime is ";
    t.printStandard();

    t.setTime(99, 99, 99); // attempt invalid settings
    cout << "\n\nAfter attempting invalid settings:\n"
        << "Military time: ";
    t.printMilitary();
    cout << "\nStandard time: ";
    t.printStandard();
    cout << endl;
    return 0;
}

```

Fig. 16.3 Puesta en práctica de tipos de datos abstractos Time como una clase (parte 2 de 3).

```

The initial military time is 00:00:00
The initial standard time is 12:00:00 AM

Military time after setTime is 13:27:06
Standard time after setTime is 1:27:06 PM

After attempting invalid settings:
Military time: 00:00:00
Standard time: 12:00:00 AM

```

Fig. 16.3 Puesta en práctica de tipos de datos abstractos Time como una clase (parte 3 de 3).

Otra vez, note que los miembros de datos **hour**, **minute** y **second** están anteceditos por la etiqueta **private**:. Los miembros de datos privados de una clase Por lo regular no son accesibles fuera de la misma. (Otra vez, veremos en el capítulo 17, que los amigos de una clase pueden tener acceso a los miembros privados de dicha clase.) La filosofía aquí es que la representación de datos reales utilizados dentro de una clase no debe importarle a los clientes de dicha clase. En este sentido, la puesta en práctica de una clase se dice que está *oculta* a sus clientes. Este *ocultamiento de la información* promueve la capacidad de modificación del programa y simplifica la percepción del cliente de una clase.

Observación de ingeniería de software 16.2

Los clientes de una clase utilizan la clase sin conocer los detalles internos de cómo está puesta en práctica dicha clase. Si se modifica la puesta en práctica de la clase (para mejorar el rendimiento, por ejemplo) no es necesario modificar los clientes de la clase. Esto facilita mucho la modificación de sistemas.

En este programa, el constructor **Time** sólo inicializa a 0 los miembros de datos (es decir, el equivalente en hora militar de 12 AM). Esto asegura que cuando es creado el objeto está en un estado consistente. Los valores inválidos no pueden ser almacenados en los miembros de datos, porque al crearse el objeto **Time** el constructor es automáticamente llamado y todos los intentos subsiguientes para modificar los miembros de datos son analizados por la función **setTime**.

Observación de ingeniería de software 16.3

Las funciones miembro por lo regular están formadas por unas pocas líneas de código, porque ninguna lógica es requerida para determinar si son válidos los miembros de datos.

Note que los miembros de datos de una clase no pueden ser inicializados donde son declarados en el cuerpo de la clase. Estos miembros de datos deberán ser inicializados por el constructor de la clase, o las funciones "set" les podrán asignar valores.

Error común de programación 16.2

Intentar inicializar de forma explícita un miembro de datos de una clase.

La función con el mismo nombre que la clase, pero precedido por un *carácter tilde* (~) se llama el *destructor* de dicha clase. El destructor se ocupa del trabajo de terminación en cada objeto de clase, antes que la memoria correspondiente al objeto sea reclamada por el sistema. En este ejemplo no se incluye un destructor. Analizaremos con mayor detalle constructores y destructores, más adelante en este capítulo, y en el capítulo 17.

Note que las funciones que la clase proporciona al mundo exterior están precedidas por la etiqueta **public**:. Las funciones públicas ponen en práctica las capacidades que la clase proporciona a sus clientes. Las funciones públicas de una clase se conocen como *interfaz* o *interfaz pública* de la clase.

Observación de ingeniería de software 16.4

Los clientes tienen acceso a la interfaz de una clase, pero no deberán tener acceso a la puesta en práctica de la clase.

La definición de clase contiene declaraciones de los miembros de datos de la clase y de las funciones miembro de la clase. Las declaraciones de funciones miembro son los prototipos de función, que fueron analizados en capítulos anteriores. Las funciones miembro pueden ser definidas dentro de una clase, pero es una práctica sana de programación definir las funciones fuera de la definición de clase.

Práctica sana de programación 16.2

Defina todas, a excepción de las más pequeñas funciones miembro, por afuera de la definición de clase. Esto ayuda a separar la interfaz de la clase de su puesta en práctica.

Note el uso del operador de *resolución de alcance binario* (::) en cada definición de función miembro, que en la figura 16.3 sigue a la definición de clase. Una vez definida una clase y sus funciones miembro declaradas, estas funciones deben ser definidas. Cada función miembro de clase puede ser definida en directo al cuerpo de la clase (en vez de incluir el prototipo de función de la clase) o la función puede ser definida después del cuerpo de la clase. Cuando una función miembro se define después de su correspondiente definición de clase, el nombre de función es antecedido por el nombre de clase y por el operador de resolución de alcance binario (::). Dado que diferentes clases pueden tener los mismos nombres de miembros, el operador de resolución de alcance “une” el nombre del miembro con el nombre de la clase, para fijar en forma única las funciones miembro de una clase en particular.

Aún cuando una función miembro declarada en una definición de clase pudiera definirse fuera de esa definición de clase, esa función miembro aún queda dentro del *alcance de la clase*, es decir su nombre es conocido sólo por otros miembros de la clase, a menos que se haga referencia a él vía un objeto de la clase, vía una referencia a un objeto de la clase, o vía un apuntador a un objeto de la clase. Diremos en breve más en relación con el alcance de la clase.

Es posible definir las funciones miembro en el cuerpo de la definición de clase. Las funciones que tengan más de una o dos líneas, se definen normalmente por afuera de la definición de clase, esto ayuda a separar la interfaz de clase de la puesta en práctica de la clase. Si se define una función miembro en una definición de clase, la función miembro quedará automáticamente en línea. Las funciones miembro definidas fuera de la definición de clase pudieran ser puestas en línea, mediante el uso explícito de la palabra reservada **inline**. Recuerde que el compilador se reserva el derecho de poner o no en línea cualquier función.

Observación de ingeniería de software 16.5

Declarar funciones miembro dentro de una definición de clase y definir dichas funciones miembro por fuera de dicha definición de clase separa la interfaz de una clase de su puesta en práctica. Esto promueve buena ingeniería de software.

Sugerencia de rendimiento 16.3

Definir una función miembro pequeña dentro de una definición de clase automática coloca la función miembro en línea (si el compilador así lo decide). Esto puede mejorar el rendimiento, pero no promueve una mejor ingeniería de software.

Resulta interesante que las funciones miembro **printMilitary** y **printStandard** no toman argumentos. Esto es debido a que las funciones miembro saben que deben imprimir los miembros de datos del objeto particular **Time** para los cuales han sido invocadas. Esto hace que las llamadas de funciones miembro sean más concisas que las llamadas de función convencionales de la programación procedural.

Observación de ingeniería de software 16.6

Usar un enfoque de programación orientado a objetos a menudo puede simplificar las llamadas de función al reducir el número de parámetros a pasarse. Este beneficio de la programación orientada a objetos se deriva del hecho que el encapsulado de los miembros de datos y las funciones miembros dentro de un objeto le da a las funciones miembro el derecho de acceso a los miembros de datos.

Las clases simplifican la programación porque el cliente (o el usuario del objeto de clase) sólo necesita preocuparse de las operaciones encapsuladas o incrustadas dentro del objeto. Dichas operaciones por lo regular están diseñadas para ser orientadas al cliente, más bien que orientadas a la puesta en práctica. Los clientes no necesitan preocuparse con la puesta en práctica de la clase. De hecho las puestas en práctica cambian, pero con menos frecuencia que las interfaces. Cuando se modifique una puesta en práctica, aquel código que sea dependiente de la puesta en práctica, deberá ser modificado. Mediante la ocultación de la puesta en práctica, eliminamos la posibilidad de que otras partes del programa se hagan dependientes de los detalles de la puesta en práctica de la clase.

A menudo, no es necesario crear clases “desde cero”. Más bien, pueden ser *derivadas*, partiendo de otras clases que contienen operaciones que las nuevas clases pueden utilizar. O las clases pueden incluir como miembros objetos de otras clases. Esta *reutilización del software* puede mejorar el rendimiento del programador de forma significativa. Se llama *herencia* derivar nuevas clases partiendo de clases existentes, y en el capítulo 19 se analizará en detalle. Incluir clases como miembros de otras clases se llama *composición*, y se analiza en el capítulo 17.

16.6 Alcance de clase y acceso a miembros de clase.

Los nombres de variables y los nombres de función declarados en una definición de clase, y los nombres de datos y funciones miembro de una clase, pertenecen al *alcance de dicha clase*. Las funciones no miembros se definen en *alcance de archivo*.

Dentro del alcance de clase, los miembros de clase son accesibles de inmediato por todas las funciones miembro de dicha clase y pueden ser referenciados sólo por su nombre. Fuera del alcance de una clase, los miembros de clase se referencian, ya sea a través del nombre del objeto, una referencia a un objeto, o un apuntador a un objeto.

Las funciones miembros de una clase pueden tener homónimas, pero sólo por funciones dentro del alcance de dicha clase. Para hacer la homonimia de una función miembro, sólo incluya en la definición de clase un prototipo para cada versión de la función homónima, y proporcione una definición de función por separado para cada una de las versiones de la función.

Error común de programación 16.3

Intentar hacer la homonimia de una función miembro mediante una función no incluida en el alcance de dicha clase.

Las funciones miembro tienen *alcance de función* dentro de una clase. Si la función miembro define una variable con el mismo nombre que una variable con alcance de clase, la variable con alcance de clase queda oculta por la variable con alcance de función, dentro del alcance de función. Se puede tener acceso a esta variable oculta mediante el operador de resolución de alcance precediendo el operador con el nombre de la clase. Se puede tener acceso a las variables globales ocultas mediante el operador de resolución de alcance unario (vea el capítulo 15).

Los operadores utilizados para tener acceso a los miembros de clase son idénticos a los operadores para tener acceso a los miembros de estructura. El *operador de selección de miembro de punto* (.) se combina con el nombre de una objeto o con una referencia a un objeto para tener acceso a los miembros del objeto. El *operador de selección de miembro de flecha* (->) se combina con un apuntador a un objeto para tener acceso a los miembros de dicho objeto.

El programa de la figura 16.4 utiliza una clase simple llamada **Count** con el miembro de datos público **x** del tipo **int**, y la función miembro pública **print** para ilustrar el acceso a los miembros de una clase mediante los operadores de selección de miembros. El programa produce tres variables del tipo **Count**: **counter**, **counterRef** (una referencia a un objeto **Count**), y **counterPtr** (un apuntador a un objeto **Count**). La variable **counterRef** se declara para hacer referencia a **counter**, y la variable **counterPtr** se declara para señalar a **counter**. Es importante hacer notar que aquí el miembro de datos **x** se ha hecho público simplemente para demostrar cómo se tiene acceso a los miembros públicos. Como hemos indicado, típicamente los datos se hacen privados como lo haremos en todos los ejemplos subsiguientes.

16.7 Cómo separar la interfaz de la puesta en práctica

Uno de los principios fundamentales de buena ingeniería de software es la separación de la interfaz de la puesta en práctica. Esto facilita la modificación de los programas. Por lo que se refiere a los clientes de una clase, los cambios en la puesta en práctica de una clase no afectan al cliente, siempre y cuando no sea modificada la interfaz de la clase.

Observación de ingeniería de software 16.7

Coloque la declaración de la clase en un archivo de cabecera a incluirse por cualquier cliente que desee utilizar dicha clase. Esto forma la interfaz pública de la clase. Coloque las definiciones de las funciones miembro de la clase en un archivo fuente. Esto conforma la puesta en práctica de la clase.

Observación de ingeniería de software 16.8

Los clientes de una clase no necesitan ver el código fuente de la clase a fin de utilizarla. Los clientes deben, sin embargo, poder tener la capacidad de enlazarse con el código objeto de la clase.

Esto alienta a los fabricantes independientes de software (ISV, por *independent software vendors*) a proporcionar bibliotecas de clase, a la venta o para su licencia. En sus productos los ISV ofrecen sólo los archivos de cabecera y los módulos de objeto. Ninguna información propietaria se da a conocer como sería el caso si se proporcionara código fuente. La comunidad de usuarios de C++ se beneficia al tener disponibles mas bibliotecas de clase, producidas por los ISV.

De hecho, las cosas no son tan color de rosa. Los archivos de cabecera sí contienen alguna porción de la puesta en práctica, así como sugerencias en relación con ella. Las funciones miembro en línea, por ejemplo, necesitan estar en un archivo de cabecera, por lo que cuando el compilador compila a un cliente, el cliente pueda incluir en su lugar la definición de función en línea. Los miembros privados son listados en la declaración de clase dentro del archivo de cabecera, por lo

```
// FIG16_4.CPP
// Demonstrating the class member access operators . and ->
//
// Caution: In future examples we will use private data.
#include <iostream.h>

// Simple class Count
class Count {
public:
    int x;
    void print() { cout << x << '\n'; }
};

main()
{
    Count counter,           // create counter object
      *counterPtr = &counter, // pointer to counter
      &counterRef = counter;  // reference to counter

    cout << "Assign 7 to x and print using the object's name: ";
    counter.x = 7;           // assign 7 to data member x
    counter.print();         // call member function print

    cout << "Assign 8 to x and print using a reference: ";
    counterRef.x = 8;        // assign 8 to data member x
    counterRef.print();      // call member function print

    cout << "Assign 10 to x and print using a pointer: ";
    counterPtr->x = 10;       // assign 10 to data member x
    counterPtr->print();      // call member function print
    return 0;
}
```

```
Assign 7 to x and print using the object's name: 7
Assign 8 to x and print using a reference: 8
Assign 10 to x and print using a pointer: 10
```

Fig. 16.4 Cómo tener acceso a los miembros de datos de un objeto y a las funciones miembro a través del nombre del objeto, a través de una referencia o a través de un apuntador al objeto.

que estos miembros quedan visibles a los clientes, aún cuando los clientes no puedan tener acceso a ellos.

Observación de ingeniería de software 16.9

Aquella información de importancia a la interfaz con una clase debería estar incluida en el archivo de cabecera. Aquella información que sólo será utilizada internamente en la clase y que no será necesaria para los clientes de la clase, debería ser incluida en el archivo fuente no publicado. Este es otra vez otro ejemplo del principio del mínimo privilegio.

En la figura 16.5 el programa de la figura 16.3 se divide en varios archivos. Al construir un programa C++, cada definición de clase por lo regular es colocado en un *archivo de cabecera*, y

las definiciones de funciones miembro de dicha clase se colocan en los *archivos de código fuente* con el mismo nombre base. Los archivos de cabecera se incluyen (mediante **#include**) en cada uno de los archivos en los cuales se utiliza la clase, y se compila el archivo fuente de las definiciones de funciones miembro y se enlaza con el archivo que contiene el programa principal. Vea la documentación correspondiente a su compilador, a fin de determinar cómo compilar y enlazar programas formados de varios archivos fuente.

La figura 16.5 está formada del archivo de cabecera **time1.h** en el cual se declara la clase **Time**, el archivo **time1.cpp** en el cual se definen las funciones miembro de la clase **Time**, y del archivo **fig16_5.cpp** en el cual se define la función **main**. La salida para este programa es idéntica a la salida de la figura 16.3.

Note que la declaración de clase se encierra en el siguiente código de preprocesador (vea el capítulo 13):

```
// prevent multiple inclusions of header file
#ifndef TIME1_H
#define TIME1_H
...
#endif
```

Cuando elaboremos programas más extensos, se colocarán también otras definiciones y declaraciones en los archivos de cabecera. Las directivas anteriores de preprocesador evitan que se incluya el código entre **#ifndef** y **#endif**, si ya ha sido definido el nombre **TIME1_H**. Si el encabezado no ha sido incluido previamente en un archivo, el nombre **TIME1_H** queda definido mediante la directiva **#define** y los enunciados del archivo de cabecera serán incluidos. Si el encabezado ha sido ya incluido, **TIME1_H** ya está definido y el archivo de cabecera no se vuelve a incluir. Nota: la regla convencional que utilizamos para el nombre de la constante simbólica en las directrices de preprocesador es sólo el nombre del archivo de cabecera, con el carácter de subrayado substituyendo el punto.

Práctica sana de programación 16.3

Utilice en un programa las directivas de preprocesador **#ifndef**, **#define** y **#endif** para evitar que los archivos de cabecera sean incluidos más de una vez.

Práctica sana de programación 16.4

Utilice el nombre del archivo de cabecera, con el punto reemplazado por un subrayado, en las directivas de preprocesador **#ifndef** y **#define** correspondientes a un archivo de cabecera.

16.8 Cómo controlar el acceso a miembros

Las etiquetas **public:** y **private:** (así como **protected:**, como veremos en el capítulo 19, “Herencia”) se utilizan para controlar el acceso a los miembros de datos y a las funciones miembro de una clase. El modo de acceso para las clases es **private:** por omisión, de tal forma que todos los miembros que aparezcan después del encabezado de clase y antes de la primera etiqueta son privados. Después de cada una de las etiquetas, el modo que fue invocado por dicha etiqueta será aplicado hasta la siguiente etiqueta, o hasta la llave derecha de terminación (**}**) de la definición de clase. Pueden repetirse las etiquetas **public:**, **private:** y **protected:**, pero este tipo de utilización es raro y puede ser confuso.

```
// TIME1.H
// Declaration of the Time class.
// Member functions are defined in TIME1.CPP

// prevent multiple inclusions of header file
#ifndef TIME1_H
#define TIME1_H

// Time abstract data type definition
class Time {
public:
    Time(); // default constructor
    void setTime(int, int, int); // set hour, minute and second
    void printMilitary(); // print military time format
    void printStandard(); // print standard time format
private:
    int hour; // 0 - 23
    int minute; // 0 - 59
    int second; // 0 - 59
};

#endif
```

Fig. 16.5 Archivo de cabecera de la clase **Time** (parte 1 de 4).

```
// TIME1.CPP
// Member function definitions for Time class.
#include <iostream.h>
#include "time1.h"

// Time constructor initializes each data member to zero.
// Ensures all Time objects start in a consistent state.
Time::Time() { hour = minute = second = 0; }

// Set a new Time value using military time.
// Perform validity checks on the data values.
// Set invalid values to zero (consistent state).
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Print Time in military format
void Time::printMilitary()
{
    cout << (hour < 10 ? "0" : "") << hour << ":"
         << (minute < 10 ? "0" : "") << minute << ":"
         << (second < 10 ? "0" : "") << second;
}
```

Fig. 16.5 Archivo fuente de las definiciones de funciones miembro de la clase **Time** (parte 2 de 4).

```
// Print time in standard format
void Time::printStandard()
{
    cout << ((hour == 0 || hour == 12) ? 12 : hour % 12)
        << ":" << (minute < 10 ? "0" : "") << minute
        << ":" << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}
```

Fig. 16.5 Archivo fuente de las definiciones de las funciones miembro de la clase **Time** (parte 3 de 4).

```
// FIG16_5.CPP
// Driver for Time1 class
// NOTE: Compile with TIME1.CPP
#include <iostream.h>
#include "time1.h"

// Driver to test simple class Time
main()
{
    Time t; // instantiate object t of class time

    cout << "The initial military time is ";
    t.printMilitary();
    cout << "\nThe initial standard time is ";
    t.printStandard();

    t.setTime(13, 27, 6);
    cout << "\n\nMilitary time after setTime is ";
    t.printMilitary();
    cout << "\nStandard time after setTime is ";
    t.printStandard();

    t.setTime(99, 99, 99); // attempt invalid settings
    cout << "\n\nAfter attempting invalid settings:\n"
        << "Military time: ";
    t.printMilitary();
    cout << "\nStandard time: ";
    t.printStandard();
    cout << endl;
    return 0;
}
```

```
The initial military time is 00:00:00
The initial standard time is 12:00:00 AM

Military time after setTime is 13:27:06
Standard time after setTime is 1:27:06 PM

After attempting invalid settings:
Military time: 00:00:00
Standard time: 12:00:00 AM
```

Fig. 16.5 Programa manejador de la clase **Time** (parte 4 de 4).

Se puede tener acceso a los miembros de clase privado sólo por miembros (y amigos) de dicha clase. Se puede tener acceso a los miembros públicos de una clase mediante cualquier función del programa.

El fin primordial de los miembros públicos es presentar a los clientes de la clase un panorama de los servicios que la clase proporciona. Este conjunto de servicios forma la *interfaz pública* de la clase. Los clientes de la clase no necesitan preocuparse de cómo la clase ejecuta sus tareas. Los miembros privados de una clase, así como las definiciones de sus funciones miembro públicas, no son accesibles a los clientes de una clase. Estos componentes forman la *puesta en práctica* de la clase.

Observación de ingeniería de software 16.10

C++ alienta a que los programas sean independientes de la puesta en práctica. Cuando se modifica la puesta en práctica de una clase utilizada mediante código independiente de la misma, dicho código no necesita ser cambiado, pero pudiera necesitar ser recopilado.

Error común de programación 16.4

Un intento por parte de una función, que no sea un miembro de una clase particular (o un amigo de dicha clase) de obtener acceso a un miembro privado de dicha clase.

En la figura 16.6, se demuestra que los miembros de clase privados son accesibles a través de la interfaz de clase pública mediante el uso de las funciones miembro públicos. Cuando se compila este programa, el compilador genera dos errores, indicando que no es accesible el miembro privado especificado en cada uno de los enunciados. La figura 16.6 incluye a `time1.h` y se compila con `time1.cpp` partiendo de la figura 16.5.

```
// FIG16_6.CPP
// Demonstrate errors resulting from attempts
// to access private class members.
#include <iostream.h>
#include "time1.h"

main()
{
    Time t;

    // Error: 'Time::hour' is not accessible
    t.hour = 7;

    // Error: 'Time::minute' is not accessible
    cout << "minute = " << t.minute;

    return 0;
}
```

```
Compiling FIG16_6.CPP:
Error FIG16_6.CPP 12: 'Time::hour' is not accessible
Error FIG16_6.CPP 15: 'Time::minute' is not accessible
```

Fig. 16.6 Intento erróneo de acceso a los miembros privados de una clase.

Práctica sana de programación 16.5

Si en una declaración de clase decide listar primero los miembros privados, utilice en forma explícita la etiqueta **private**: a pesar de que por omisión se supone que **private**: es asumido. Esto mejora la claridad del programa. Nuestra preferencia es listar primero los miembros **public**: de una clase.

Práctica sana de programación 16.6

A pesar del hecho que las etiquetas **public**: y **private**: pueden ser repetidas y entremezcladas, liste primero en un grupo todos los miembros públicos de una clase y a continuación liste en otro grupo todos los miembros privados. Esto enfoca la atención del cliente en la interfaz pública de la clase, en vez de en la puesta en práctica de la misma.

Observación de ingeniería de software 16.11

Conserve privados todos los miembros de datos de una clase. Proporcione funciones miembro públicas para definir los valores de los miembros de datos privados y para obtener los valores de los miembros de datos privados. Esta arquitectura ayuda a ocultar la puesta en práctica de una clase a la vista de sus clientes, lo que reduce errores y mejora la capacidad de modificación del programa.

El cliente de una clase puede ser una función miembro de otra clase, o puede ser una función global.

Para miembros de una clase el acceso por omisión es privado. El acceso a miembros de una clase puede ser definido en forma explícita a protegido o a público. El acceso por omisión correspondiente a miembros **struct** y a miembros de unión es **public**. El acceso a miembros de un **struct** puede ser definido en forma explícita a **public** o a **private** (o a **protected**, como veremos en el capítulo 19). Tratándose de una unión, los especificadores de acceso de miembros no pueden ser utilizados en forma explícita.

Observación de ingeniería de software 16.12

Los diseñadores de clase utilizan miembros privados, protegidos y públicos para obligar a la idea de ocultamiento de información y al principio del mínimo privilegio.

Note que los miembros de una clase son privados por omisión, por lo que nunca es necesario utilizar de forma explícita el especificador de acceso de miembro **private**: Muchos programadores prefieren, sin embargo, listar primero la interfaz a una clase (es decir, los miembros públicos de la clase); a continuación se listan los miembros privados, con lo que se requiere el uso explícito del especificador de acceso de miembros **private**: dentro de la definición de clase.

Práctica sana de programación 16.7

Evita confusión usar los especificadores de acceso de miembro **public**:, **protected**:, y **private**: una vez en cada definición de clase.

El simple hecho que los datos de clase sean **private** no necesariamente significa que los clientes no puedan efectuar modificaciones a dichos datos. Los datos pueden ser modificados por funciones miembro o amigos de dicha clase. Como veremos, dichas funciones deberán ser diseñadas para asegurar la integridad de los datos.

El acceso a los datos privados de una clase puede ser controlado cuidadosamente mediante el uso de las funciones miembro, llamadas *funciones de acceso*. Por ejemplo, para permitir a los clientes leer el valor de datos privados, la clase puede proporcionar una función “get”. Para

permitir que los clientes modifiquen datos privados, la clase puede proporcionar una función “set”. Una modificación como ésta parecería violar el concepto de datos privados. Pero una función miembro **set** puede proporcionar capacidades de validación de datos (como la verificación de rangos) para asegurarse que el valor se define de forma correcta. Una función **get** no necesita exponer los datos en formato “en bruto”; más bien la función **get** puede editar los datos y limitar la parte de dichos datos que el cliente verá.

Observación de ingeniería de software 16.13

Hacer privados los miembros de datos de una clase y públicas las funciones miembro de la clase, facilita la depuración, porque los problemas con las manipulaciones de datos se localizan ya sea en las funciones miembros de la clase o en los amigos de la misma

16.9 Funciones de acceso y funciones de utilidad

No todas las funciones miembro necesitan ser públicas para servir como parte de la interfaz de una clase. Algunas funciones miembro se conservan privadas, y sirven como funciones de utilidad para otras funciones de la clase.

Observación de ingeniería de software 16.14

Las funciones miembro tienen tendencia a agruparse en varias categorías diferentes: funciones que leen y regresan el valor de miembros de datos privados, funciones que definen el valor de miembros de datos privados, funciones que ponen en práctica las características de la clase, y funciones que ejecutan varias tareas mecánicas para la clase, como la inicialización de objetos de clase, la asignación de objetos de clase, la conversión entre clases y tipos incorporados o entre clases y otras clases, y el manejo de memoria para objetos de clase.

Las funciones de acceso pueden leer o desplegar datos. Otro uso común de las funciones de acceso es probar la veracidad o falsedad de condiciones —dichas funciones a menudo se conocen como *funciones predicadas*. Un ejemplo de una función predicada sería una función **isEmpty** para cualquier clase contenedor, como es una lista enlazada, una pila o una cola. Un programa probaría **isEmpty** antes de intentar leer cualquier otro elemento del objeto contenedor. Una función predicada **isFull** pudiera probar un objeto de clase contenedor para determinar si ya no tiene espacio adicional.

En la figura 16.7 se demuestra el concepto de una *función de utilidad*. Una función de utilidad no forma parte de la interfaz de una clase. Más bien, es una función miembro privada, que da apoyo a la operación de las funciones miembro públicas de la clase. Las funciones de utilidad no están concebidas para ser utilizadas por los clientes de una clase.

La clase **SalesPerson** tiene un arreglo de 12 cifras de ventas mensuales inicializadas a cero por el constructor y definidas mediante la función **setSales** en valores proporcionados por el usuario. La función miembro pública **printAnnualSales** imprime las ventas totales correspondientes a los 12 últimos meses. La función de utilidad **totalAnnualSales** totaliza las 12 cifras de ventas mensuales, para uso y beneficio de **printAnnualSales**. La función miembro **printAnnualSales** edita las cifras de ventas y las coloca en formato de moneda. En el capítulo 21 se explica con detalle cada una de las capacidades de formato de C++ utilizadas aquí. Por ahora, sólo damos una breve explicación. La llamada

```
setprecision(2)
```

indica que serán impresos dos dígitos de “precisión” a la derecha del punto decimal, exactamente como necesitamos para cantidades en moneda, tales como 23.47. Esta llamada se conoce como

un “manipulador de flujo parametrizado”. Un programa que utilice este tipo de llamadas, también deberá contener la directiva de preprocesador

```
#include <iomanip.h>
```

La línea

```
setiosflags(ios::fixed | ios::showpoint);
```

hace que se muestre la cifra de ventas en el formato conocido como de punto fijo (a diferencia de la notación científica). La opción **showpoint** obliga a que aparezca el punto decimal y los ceros después del punto, aún si el número es una cantidad redonda de dólares, como 47.00. En C++, de no haberse definido la opción **showpoint**, un número como éste se imprimiría solo como 47.

16.10 Cómo inicializar objetos de clase: constructores

Después de que los objetos son creados, sus miembros pueden ser inicializados mediante funciones *constructor*. Un constructor es una función miembro de clase con el mismo nombre que la clase. El programador proporciona el constructor el cual entonces, cada vez que se crea un objeto de dicha clase, es invocado automáticamente. *Los miembros de datos de una clase no pueden ser inicializados en la definición de clase.* Más bien, los miembros de datos deben ser inicializados en un constructor de la clase o sus valores definidos más adelante después de que el objeto haya sido creado. Los constructores no pueden especificar tipos de regreso y valores de regreso. Los constructores pueden ser sujetos de homonimia, para permitir una variedad de maneras de inicializar objetos de una clase.

Error común de programación 16.5

Intentar inicializar explícitamente un miembro de datos de una clase dentro de la definición de clase.

```
// SALESP.H
// SalesPerson class definition
// Member functions defined in SALESP.CPP

#ifndef SALESP_H
#define SALESP_H

class SalesPerson {
public:
    SalesPerson();           // constructor
    void setSales();         // user supplies sales figures
    void printAnnualSales();

private:
    double sales[13];        // 12 monthly sales figures
    double totalAnnualSales(); // utility function
};

#endif
```

Fig. 16.7 Cómo utilizar una función de utilidad (parte 1 de 3).

```
// SALESP.CPP
// Member functions for class SalesPerson
#include <iostream.h>
#include <iomanip.h>
#include "salesp.h"

// Constructor function initializes array
SalesPerson::SalesPerson()
{
    for (int i = 0; i <= 12; i++)
        sales[i] = 0.0;
}

// Function to set the 12 monthly sales figures
void SalesPerson::setSales()
{
    for (int i = 1; i <= 12; i++) {
        cout << "Enter sales amount for month "
              << i << ": ";
        cin >> sales[i];
    }
}

// Private utility function to total annual sales
double SalesPerson::totalAnnualSales()
{
    double total = 0.0;

    for (int i = 1; i <= 12; i++)
        total += sales[i];

    return total;
}

// Print the total annual sales
void SalesPerson::printAnnualSales()
{
    cout << setprecision(2)
          << setiosflags(ios::fixed | ios::showpoint)
          << "\nThe total annual sales are: $"
          << totalAnnualSales() << endl;
}
```

Fig. 16.7 Cómo utilizar una función de utilidad (parte 2 de 3).

Error común de programación 16.6

Intentar declarar un tipo de regreso para un constructor, y/o intentar regresar un valor proveniente de un constructor.

Práctica sana de programación 16.8

Cuando sea apropiado (casi siempre), proporcione un constructor para asegurarse que todo objeto es inicializado correctamente, con valores significativos.

```
// FIG16_7.CPP
// Demonstrating a utility function
// Compile with SALESP.CPP
#include "salesp.h"

main()
{
    SalesPerson s;          // create SalesPerson object s

    s.setSales();
    s.printAnnualSales();

    return 0;
}
```

```
Enter sales amount for month 1: 5314.76
Enter sales amount for month 2: 4292.38
Enter sales amount for month 3: 4589.83
Enter sales amount for month 4: 5534.03
Enter sales amount for month 5: 4376.34
Enter sales amount for month 6: 5698.45
Enter sales amount for month 7: 4439.22
Enter sales amount for month 8: 5893.57
Enter sales amount for month 9: 4909.67
Enter sales amount for month 10: 5123.45
Enter sales amount for month 11: 4024.97
Enter sales amount for month 12: 5923.92

Total annual sales: $60120.59
```

Fig. 16.7 Cómo utilizar una función de utilería (parte 3 de 3).

Práctica sana de programación 16.9

Cada función miembro (y amigo) que modifique los miembros de datos privados de un objeto deberán asegurarse que los datos se conserven en un estado consistente.

Cuando se declara un objeto de una clase, se pueden dar *inicializadores* en paréntesis a la derecha del nombre del objeto y antes del punto y coma. Estos inicializadores son pasados como argumentos al constructor de la clase. Veremos pronto varios ejemplos de estas *llamadas de constructor*.

16.11 Cómo utilizar argumentos por omisión con los constructores

El constructor correspondiente a `time1.cpp` (figura 16.5) inicializó a 0 `hour`, `minute` y `second` (es decir, a las 12 de media noche en hora militar). Los constructores pueden contener argumentos por omisión. La figura 16.8 vuelve a definir la función constructor `Time` para incluir para cada variable argumentos por omisión iguales a cero. Al dar al constructor argumentos por omisión, aun si a una llamada de constructor no se dan valores, el objeto está garantizado de estar en un estado consistente, debido a los argumentos por omisión. Un constructor suministrado

por el programador, y que utiliza sus argumentos por omisión, también se llama un constructor por omisión (puede ser invocado sin argumentos). El constructor utiliza el mismo tipo de código de validación que `setTime`, para asegurarse que el valor suministrado correspondiente a `hour` está en el rango de 0 a 23, y que los valores correspondientes a `minute` y `second` son cada uno de ellos dentro del rango 0 a 59. Si un valor está fuera de rango, se define a cero (este es un ejemplo de cómo asegurar que un miembro de datos se conserva en un estado consistente). El constructor podría llamar directamente a `setTime`, pero esto crearía la sobrecarga de una llamada de función adicional. El programa de la figura 16.8 inicializa cinco objetos `Time` uno con los tres argumentos en sus valores por omisión en la llamada de constructor, uno con un argumento definido, uno con dos definidos, uno con tres definidos y uno con tres argumentos inválidos especificados. Aparece mostrado el contenido de cada uno de los miembros de datos del objeto, después de la ejemplificación y de la inicialización.

Si para una clase no se define ningún constructor, el compilador creará un constructor por omisión. Dicho constructor no ejecutará ninguna inicialización, por lo que no se garantizará que esté en un estado consistente.

Práctica sana de programación 16.10

Incluya siempre un constructor que ejecute la inicialización adecuada para su clase.

16.12 Cómo utilizar destructores

Un *destructor* es una función miembro especial de una clase. El nombre del destructor para una clase es la *tilde* (~) seguida por el nombre de la clase. Esta regla convencional de identificación tiene un atractivo intuitivo, porque el operador de tilde es el operador de complemento a nivel de bits, y, en cierto sentido, el destructor es el complemento del constructor.

```
// TIME2.H
// Declaration of the Time class.
// Member functions defined in TIME2.CPP

// prevent multiple inclusions of header file
#ifndef TIME2_H
#define TIME2_H

class Time {
public:
    Time(int = 0, int = 0, int = 0); // default constructor
    void setTime(int, int, int);
    void printMilitary();
    void printStandard();
private:
    int hour;
    int minute;
    int second;
};

#endif
```

Fig. 16.8 Cómo usar un constructor mediante argumentos por omisión (parte 1 de 4).

```
// TIME2.CPP
// Member function definitions for Time class.
#include "time2.h"
#include <iostream.h>

// Constructor function to initialize private data.
// Default values are 0 (see class definition).
Time::Time(int hr, int min, int sec)
{
    hour = (hr >= 0 && hr < 24) ? hr : 0;
    minute = (min >= 0 && min < 60) ? min : 0;
    second = (sec >= 0 && sec < 60) ? sec : 0;
}

// Set values of hour, minute, and second.
// Invalid values are set to 0.
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Display time in military format: HH:MM:SS
void Time::printMilitary()
{
    cout << (hour < 10 ? "0" : "") << hour << ":"
        << (minute < 10 ? "0" : "") << minute << ":"
        << (second < 10 ? "0" : "") << second;
}

// Display time in standard format: HH:MM:SS AM (or PM)
void Time::printStandard()
{
    cout << ((hour == 0 || hour == 12) ? 12 : hour % 12)
        << ":" << (minute < 10 ? "0" : "") << minute
        << ":" << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}
```

Fig. 16.8 Cómo usar un constructor con argumentos por omisión (parte 2 de 4).

Un destructor de clase es llamado automáticamente cuando un objeto de una clase se sale de alcance. De hecho el destructor mismo en realidad no destruye el objeto, más bien ejecuta *trabajos de terminación*, antes de que el sistema recupere el espacio de memoria del objeto con el fin de que pueda ser utilizado para almacenar nuevos objetos.

Un destructor no recibe ningún parámetro ni regresa ningún valor. Una clase sólo puede tener un destructor —la homonimia de destructores no está permitida.

Error común de programación 16.7

Intentar pasar argumentos a un destructor, regresar valores de un destructor, o hacer la homonimia de un destructor.

```
// FIG16_8.CPP
// Demonstrating a default constructor
// function for class Time.
#include <iostream.h>
#include "time2.h"

main()
{
    Time t1, t2(2), t3(21, 34), t4(12, 25, 42),
        t5(27, 74, 99);

    cout << "Constructed with:\n"
        << "all arguments defaulted:\n ";
    t1.printMilitary();
    cout << "\n ";
    t1.printStandard();

    cout << "\nhour specified; minute and second defaulted:\n ";
    t2.printMilitary();
    cout << "\n ";
    t2.printStandard();

    cout << "\nhour and minute specified; second defaulted:\n ";
    t3.printMilitary();
    cout << "\n ";
    t3.printStandard();

    cout << "\nhour, minute, and second specified:\n ";
    t4.printMilitary();
    cout << "\n ";
    t4.printStandard();

    cout << "\nall invalid values specified:\n ";
    t5.printMilitary();
    cout << "\n ";
    t5.printStandard();
    cout << endl;

    return 0;
}
```

Fig. 16.8 Cómo usar un constructor con argumentos por omisión (parte 3 de 4).

Note que no hemos proporcionado destructores para las clases que hasta ahora hemos presentado. De hecho, con clases sencillas rara vez se utilizan destructores. En el capítulo 18, veremos que los destructores son apropiados para aquellas clases cuyos objetos contienen almacenamiento dinámicamente asignado (por ejemplo, para arreglos y cadenas).

16.13 Cuándo son llamados los destructores y los constructores

Por lo regular, los constructores y los destructores son llamados de forma automática. El orden en el cual son hechas estas llamadas de función, depende del orden en el cual los objetos entran y salen de alcance. En general, las llamadas de destructor se efectúan en orden inverso a las

```

Constructed with:
all arguments defaulted:
    00:00:00
    12:00:00 AM
hour specified; minute and second defaulted:
    02:00:00
    2:00:00 AM
hour and minute specified; second defaulted:
    21:34:00
    9:34:00 PM
hour, minute, and second specified:
    12:25:42
    12:25:42 PM
all invalid values specified:
    00:00:00
    12:00:00 AM

```

Fig. 16.8 Cómo usar un constructor con argumentos por omisión (parte 4 de 4).

llamadas de constructor. Sin embargo, la duración de almacenamiento (persistencia) de los objetos puede modificar el orden en el cual los destructores son llamados.

Se llaman a los constructores para objetos declarados en alcance global al principio de la ejecución del programa. Los destructores correspondientes son llamados a la terminación del programa.

Los constructores son llamados para objetos locales automáticos, cuando estos objetos son declarados. Los destructores correspondientes son llamados cuando los objetos salen de alcance (es decir, cuando salen del bloque en el cual están declarados). Note que tanto los constructores como los destructores, correspondientes a los objetos locales automáticos, podrían ser llamados muchas veces, conforme los objetos entren y salgan de alcance.

Para los objetos locales estáticos los constructores son llamados una vez cuando estos objetos son declarados. Los destructores correspondientes son llamados a la terminación del programa.

El programa de la figura 16.9 demuestra el orden en el que se llaman los constructores y los destructores para objetos del tipo **CreateAndDestroy** en varios alcances. El programa declara **first** en alcance global. Su constructor es llamado conforme el programa inicia ejecución y su destructor es llamado a la terminación del programa, después de que todos los demás objetos han sido destruidos.

La función **main** declara tres objetos. Los objetos **second** y **fourth** son objetos automáticos locales, y el objeto **third** es un objeto local estático. Los constructores correspondientes a cada uno de estos objetos son llamados cuando es declarado cada uno de estos objetos. Cuando se llega al final de **main** los destructores para los objetos **fourth** y **second** son llamados en ese orden. Dado que el objeto **third** es estático, existirá desde el momento en que se declara, hasta la terminación del programa. El destructor para el objeto **third** es llamado antes del destructor para **first**, pero después de que hayan sido destruidos todos los demás objetos.

La función **create** declara tres objetos. Los objetos **fifth** y **seventh** son objetos automáticos locales, y el objeto **sixth** es un objeto local estático. Cuando se llega al final de **create** los destructores correspondientes a los objetos **seventh** y **fifth** son llamados en ese orden. Debido a que el objeto **sixth** es estático, existe desde el punto en el cual es declarado,

```

// CREATE.H
// Definition of class CreateAndDestroy.
// Member functions defined in CREATE.CPP.

#ifndef CREATE_H
#define CREATE_H

class CreateAndDestroy {
public:
    CreateAndDestroy(int); // constructor
    ~CreateAndDestroy();   // destructor
private:
    int data;
};

#endif

```

Fig. 16.9 Cómo demostrar el orden en el cual son llamados los constructores y los destructores (parte 1 de 3).

```

// CREATE.CPP
// Member function definitions for class CreateAndDestroy
#include <iostream.h>
#include "create.h"

CreateAndDestroy::CreateAndDestroy(int value)
{
    data = value;
    cout << "Object " << data << "   constructor";
}

CreateAndDestroy::~~CreateAndDestroy()
{ cout << "Object " << data << "   destructor " << endl; }

```

Fig. 16.9 Cómo demostrar el orden en el cual son llamados los constructores y los destructores (parte 2 de 3).

hasta la terminación del programa. El destructor para el objeto **sixth** es llamado antes de los destructores correspondientes a **third** y a **first**, pero después de que todos los otros objetos hayan sido destruidos.

16.14 Cómo utilizar miembros de datos y funciones miembro

Los miembros de datos privados pueden ser manipulados sólo por funciones miembro y los amigos de la clase. Una manipulación típica pudiera ser el ajuste del saldo bancario de un cliente (por ejemplo, un miembro de dato privado de la clase **BankAccount**), hecho por una función miembro **computeInterest**.

A menudo las clases proporcionan funciones miembro públicas para permitir a los clientes de la clase definir (es decir, escribir) u obtener (es decir, leer) los valores de los miembros de datos privados. Estas funciones no necesitan ser en específico funciones "set" y "get", pero a menudo lo son. En específico, una función miembro, que define el miembro de datos **interestRate**

```
// FIG16_9.CPP
// Demonstrating the order in which constructors and
// destructors are called.
#include <iostream.h>
#include "create.h"

void create(void); // prototype
CreateAndDestroy first(1); // global object

main()
{
    cout << "    (global created before main)\n";
    CreateAndDestroy second(2); // local object
    cout << "    (local automatic in main)\n";

    static CreateAndDestroy third(3); // local object
    cout << "    (local static in main)\n";

    create(); // call function to create objects
    CreateAndDestroy fourth(4); // local object
    cout << "    (local automatic in main)\n";
    return 0;
}

// Function to create objects
void create(void)
{
    CreateAndDestroy fifth(5);
    cout << "    (local automatic in create)\n";

    static CreateAndDestroy sixth(6);
    cout << "    (local static in create)\n";

    CreateAndDestroy seventh(7);
    cout << "    (local automatic in create)\n";
}
```

```
Object 1 constructor (global created before main)
Object 2 constructor (local automatic in main)
Object 3 constructor (local static in main)
Object 5 constructor (local automatic in create)
Object 6 constructor (local static in create)
Object 7 constructor (local automatic in create)
Object 7 destructor
Object 5 destructor
Object 4 constructor (local automatic in main)
Object 4 destructor
Object 2 destructor
Object 6 destructor
Object 3 destructor
Object 1 destructor
```

Fig. 16.9 Cómo demostrar el orden en que se llaman constructores y destructores (parte 3 de 3).

típicamente se llamaría `setInterestRate`, y una función miembro que obtiene el `interestRate`, típicamente sería llamada `getInterestRate`. Las funciones `get` también por lo común son conocidas como funciones “de consulta”.

Parecería que proporcionar capacidades tanto de definir como de obtener resultaría en esencial lo mismo que el hacer públicos los miembros de datos. Esto es otra vez otra sutileza de C++, que hace que este lenguaje sea tan atractivo para la ingeniería de software. Si un miembro de datos es público, entonces el miembro de datos puede ser leído o escrito a voluntad por cualquier función del programa. Si un miembro de datos es privado, una función “`get`” pública parecería que podría permitir a otras funciones leer los datos a voluntad, pero la función `get` controlaría el formato y la exhibición de los datos. Una función pública “`set`” podría —y muy probablemente haría— escrutinizar con cuidado cualquier intento de modificación del valor del miembro de dato. Esto garantizaría que el nuevo valor es apropiado para ese elemento de dato. Por ejemplo, serían rechazados intentos de definir el día del mes a 37, intentos de definir el peso de una persona a un valor negativo, intentos de definir una cantidad numérica a un valor alfabético, intentos de definir una calificación de un examen como 185 (cuando el rango correcto es de 0 a 100), y así en lo sucesivo.

Observación de ingeniería de software 16.15

Hacer los miembros de datos privados y controlando su acceso, especialmente el acceso de escritura, a aquellos miembros de datos vía funciones miembro, ayuda a asegurar la integridad de los datos.

Los beneficios de la integridad de los datos no son automáticos sólo porque son hechos privados los miembros de datos, el programador debe de proveer la verificación de validez. C++, sin embargo, sí proporciona una estructura en la cual los programadores pueden diseñar mejores programas de una forma conveniente.

Práctica sana de programación 16.11

Las funciones miembro que definen los valores de los datos privados, deberán verificar que los nuevos valores propuestos son correctos; si no lo son, las funciones `set` deben colocar los miembros de datos privados en un estado consistente apropiado.

En la figura 16.10 se extiende nuestra clase `Time` a fin de incluir las funciones `get` y `set` para los miembros de datos privados `hour`, `minute` y `second`. Las funciones `set` controlan de forma estricta la definición de los miembros de datos. Cualquier intento de definir cualquier miembro de datos a un valor incorrecto, hará que el miembro de dato se especifique a cero (y dejando por lo tanto dicho miembro de dato en un estado consistente). Cada función `get` sólo regresa el valor apropiado del miembro de dato. El programa primero utiliza las funciones `set` para definir los miembros de dato privados del objeto `t` de `Time` a valores válidos, y a continuación utiliza las funciones `get` para recuperar dichos valores para su extracción. A continuación las funciones `set` intentan definir los miembros `hour` y `second` a valores inválidos y el miembro `minute` a un valor válido, y a continuación las funciones `get` recuperan los valores para su extracción. La salida confirma que los valores inválidos hacen que los miembros de datos se ajusten a cero. Por último, el programa especifica la hora a 11:58:00 e incrementa el valor de minutos en 3 con una llamada a la función `incrementMinutes`. La función `incrementMinutes` es una función no miembro, que utiliza las funciones `get` y `set` para incrementar de forma correcta el miembro `minute`. Aunque esto funciona, al emitir varias llamadas de función incurre en sobrecarga de rendimiento. En el siguiente capítulo analizamos el concepto de las funciones amigas, como un medio de eliminar esta sobrecarga de rendimiento.

```

// TIME3.H
// Declaration of the Time class.
// Member functions defined in TIME3.CPP

// prevent multiple inclusions of header file
#ifndef TIME3_H
#define TIME3_H

class Time {
public:
    Time(int = 0, int = 0, int = 0); // constructor

    // set functions
    void setTime(int, int, int); // set hour, minute, second
    void setHour(int); // set hour
    void setMinute(int); // set minute
    void setSecond(int); // set second

    // get functions
    int getHour(); // return hour
    int getMinute(); // return minute
    int getSecond(); // return second

    void printMilitary(); // output military time
    void printStandard(); // output standard time
private:
    int hour; // 0 - 23
    int minute; // 0 - 59
    int second; // 0 - 59
};

#endif

```

Fig. 16.10 Declaración de la clase `Time` (parte 1 de 5).

La disponibilidad de las funciones set en una clase, proporciona cierto rango de flexibilidad en el diseño de constructores.

Error común de programación 16.8

Un constructor puede llamar a otras funciones miembro de la clase como son las funciones set o get, pero dado que el constructor está inicializando el objeto, los miembros de datos pudieran no estar aún en un estado consistente. Puede causar errores utilizar miembros de datos antes de que hayan sido inicializados de manera adecuada.

Desde un punto de vista de ingeniería de software, las funciones set son ciertamente de importancia, porque pueden llevar a cabo verificación de validez. Las funciones set y get tienen otra ventaja importante de ingeniería de software.

Observación de ingeniería de software 16.16

Tener acceso a datos privados mediante las funciones miembro set y get no sólo protege los miembros de datos contra la recepción de valores inválidos, sino también aísla a los clientes de la clase de la representación de los miembros de datos. Entonces, si por alguna razón cambia la representación de los datos (típicamente para reducir la cantidad de almacenamiento requerido o para mejorar el rendimiento), sólo las funciones miembro necesitan modificarse —no es necesario que los clientes cambien, en tanto la interfaz proporcionada por las funciones miembro se conserve igual. Los clientes pueden, sin embargo, requerir ser recompilados.

```

// TIME3.CPP
// Member function definitions for Time class.

#include "time3.h"
#include <iostream.h>

// Constructor function to initialize private data.
// Default values are 0 (see class definition).
Time::Time(int hr, int min, int sec)
{
    hour = (hr >= 0 && hr < 24) ? hr : 0;
    minute = (min >= 0 && min < 60) ? min : 0;
    second = (sec >= 0 && sec < 60) ? sec : 0;
}

// Set the values of hour, minute, and second.
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Set the hour value
void Time::setHour(int h)
{
    hour = (h >= 0 && h < 24) ? h : 0;
}

// Set the minute value
void Time::setMinute(int m)
{
    minute = (m >= 0 && m < 60) ? m : 0;
}

// Set the second value
void Time::setSecond(int s)
{
    second = (s >= 0 && s < 60) ? s : 0;
}

// Get the hour value
int Time::getHour() { return hour; }

// Get the minute value
int Time::getMinute() { return minute; }

// Get the second value
int Time::getSecond() { return second; }

// Display military format time: HH:MM:SS
void Time::printMilitary()
{
    cout << (hour < 10 ? "0" : "") << hour << ":" <<
        (minute < 10 ? "0" : "") << minute << ":" <<
        (second < 10 ? "0" : "") << second;
}

```

Fig. 16.10 Definiciones de funciones miembro para la clase `Time` (parte 2 de 5).

```
// Display standard format time: HH:MM:SS AM (or PM)
void Time::printStandard()
{
    cout << ((hour == 12) ? 12 : hour % 12) << ":"
        << (minute < 10 ? "0" : "") << minute << ":"
        << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}
```

Fig. 16.10 Definiciones de funciones miembro para la clase `Time` (parte 3 de 5).

```
// FIG16_10.CPP
// Demonstrating the Time class set and get functions
#include <iostream.h>
#include "time3.h"

main()
{
    Time t;
    void incrementMinutes(Time &, const int);

    t.setHour(17);
    t.setMinute(34);
    t.setSecond(25);

    cout << "Result of setting all valid values:\n Hour: "
        << t.getHour()
        << " Minute: " << t.getMinute()
        << " Second: " << t.getSecond() << "\n\n";

    t.setHour(234); // invalid hour set to 0
    t.setMinute(43);
    t.setSecond(6373); // invalid second set to 0

    cout << "Result of attempting to set invalid hour and"
        << " second:\n Hour: " << t.getHour()
        << " Minute: " << t.getMinute()
        << " Second: " << t.getSecond() << "\n\n";

    t.setTime(11, 58, 0);
    incrementMinutes(t, 3);

    return 0;
}
```

Fig. 16.10 Cómo utilizar las funciones set y get (parte 4 de 5).

16.15 Una trampa sutil: cómo regresar una referencia a un miembro de datos privado

Una referencia a un objeto es un seudónimo del objeto mismo y por lo tanto puede ser utilizado del lado izquierdo de un enunciado de asignación. En este contexto, la referencia es un valor a la izquierda (*lvalue*) es muy aceptable, que puede recibir un valor. Una forma de aprovechar esta

```
void incrementMinutes(Time &tt, const int count)
{
    cout << "Incrementing minute " << count
        << " times:\nStart time: ";
    tt.printStandard();

    for (int i = 1; i <= count; i++) {
        tt.setMinute((tt.getMinute() + 1) % 60);

        if (tt.getMinute() == 0)
            tt.setHour((tt.getHour() + 1) % 24);

        cout << "\nminute + 1: ";
        tt.printStandard();
    }

    cout << endl;
}
```

```
Result of setting all valid values:
Hour: 17 Minute: 34 Second: 25

Result of attempting to set invalid hour and second:
Hour: 0 Minute: 43 Second: 0

Incrementing minute 3 times:
Start time: 11:58:00 AM
minute + 1: 11:59:00 AM
minute + 1: 12:00:00 PM
minute + 1: 12:01:00 PM
```

Fig. 16.10 Cómo utilizar funciones set y get (parte 5 de 5).

capacidad (¡desafortunadamente!) es hacer que una función miembro pública de una clase regrese una referencia a un miembro de datos privado de la misma clase.

En la figura 16.11 se utiliza una versión simplificada de la clase `Time`, para demostrar el regreso de una referencia a un miembro de datos privado. Tal regreso, de hecho efectúa una llamada a la función `badSetHour` ¡un seudónimo del miembro de datos privado `hour`! La llamada de función puede ser utilizada de cualquiera de las formas en que un miembro de datos privado pueda ser utilizado, ¡incluye un valor a la izquierda (*lvalue*) en un enunciado de asignación!

Práctica sana de programación 16.12

Nunca haga que una función miembro pública regrese una referencia no `const` (o un apuntador) a un miembro de datos privado. Regresar una referencia como ésta viola el encapsulado de la clase.

El programa empieza declarando el objeto `t` de `Time` y la referencia `hourRef` que está asignada a la referencia regresada por la llamada `t.badSetHour(20)`. El programa muestra el valor del seudónimo `hourRef`. A continuación, se utiliza el seudónimo para definir el valor de `hour` a 30 (un valor inválido) y se vuelve a mostrar el valor. Finalmente, se utiliza la llamada de función misma, como un valor a la izquierda (*lvalue*) y se le asigna el valor 74 (otro valor inválido), y este valor es mostrado.

```
// TIME4.H
// Declaration of the Time class.
// Member functions defined in TIME4.CPP

// prevent multiple inclusions of header file
#ifndef TIME4_H
#define TIME4_H

class Time {
public:
    Time(int = 0, int = 0, int = 0);
    void setTime(int, int, int);
    int getHour();
    int &badSetHour(int); // DANGEROUS reference return
private:
    int hour;
    int minute;
    int second;
};

#endif
```

Fig. 16.11 Cómo regresar una referencia a un miembro de datos privado (parte 1 de 3).

```
// TIME4.CPP
// Member function definitions for Time class.
#include "time4.h"
#include <iostream.h>

// Constructor function to initialize private data.
// Calls member function setTime to set variables.
// Default values are 0 (see class definition).
Time::Time(int hr, int min, int sec)
{ setTime(hr, min, sec); }

// Set the values of hour, minute, and second.
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Get the hour value
int Time::getHour() { return hour; }

// BAD PROGRAMMING PRACTICE:
// Returning a reference to a private data member.
int &Time::badSetHour(int hh)
{
    hour = (hh >= 0 && hh < 24) ? hh : 0;
    return hour; // DANGEROUS reference return
}
```

Fig. 16.11 Cómo regresar una referencia a un miembro de datos privado (parte 2 de 3).

```
// FIG16_11.CPP
// Demonstrating a public member function that
// returns a reference to a private data member.
// Time class has been trimmed for this example.

#include <iostream.h>
#include "time4.h"

main()
{
    Time t;
    int &hourRef = t.badSetHour(20);

    cout << "Hour before modification: "
         << hourRef << '\n';
    hourRef = 30; // modification with invalid value
    cout << "Hour after modification: "
         << t.getHour() << '\n';

    // Dangerous: Function call that returns
    // a reference can be used as an lvalue.
    t.badSetHour(12) = 74;
    cout << "\n*****\n"
         << "BAD PROGRAMMING PRACTICE!!!!!!\n"
         << "badSetHour as an lvalue, Hour: "
         << t.getHour()
         << "\n*****\n";

    return 0;
}
```

```
Hour before modification: 20
Hour after modification: 30

*****
BAD PROGRAMMING PRACTICE!!!!!!
badSetHour as an lvalue, Hour: 74
*****
```

Fig. 16.11 Cómo regresar una referencia a un miembro de datos privado (parte 3 de 3).

16.16 Asignación por omisión en copia a nivel de miembro

El operador de asignación (=) es utilizado para asignar un objeto a otro objeto del mismo tipo. Dicha asignación se lleva normalmente a cabo mediante la *copia a nivel de miembro* —cada miembro de un objeto es copiado en forma individual al mismo miembro dentro de otro objeto (vea la figura 16.12). (Nota: la copia a nivel de miembro puede causar problemas serios, cuando sea utilizada con una clase cuyos miembros de datos contengan almacenamiento dinámicamente asignado; en el capítulo 18, “Homonomia de operadores”, analizaremos estos problemas y mostraremos cómo resolverlos.)

```
// FIG16_12.CPP
// Demonstrating that class objects can be assigned
// to each other using default memberwise copy
#include <iostream.h>

// Simple Date class
class Date {
public:
    Date(int = 1, int = 1, int = 1990); // default constructor
    void print();
private:
    int month;
    int day;
    int year;
};

// Simple Date constructor with no range checking
Date::Date(int m, int d, int y)
{
    month = m;
    day = d;
    year = y;
}

// Print the Date in the form mm-dd-yyyy
void Date::print()
{ cout << month << '-' << day << '-' << year; }

main()
{
    Date d1(7, 4, 1993), d2; // d2 defaults to 1/1/90

    cout << "d1 = ";
    d1.print();
    cout << "\nd2 = ";
    d2.print();

    d2 = d1; // assignment by default memberwise copy
    cout << "\nAfter default memberwise copy, d2 = ";
    d2.print();
    cout << endl;

    return 0;
}
```

```
d1 = 7-4-1993
d2 = 1-1-1990
```

```
After default memberwise copy, d2 = 7-4-1993
```

Fig. 16.12 Cómo asignar un objeto a otro mediante la copia a nivel de miembro por omisión.

Los objetos pueden ser pasados como argumentos de función y pueden ser regresados de las funciones. Este pasar y regresar se ejecuta por omisión — en llamada por valor se pasa o se regresa una copia del objeto (presentamos varios ejemplos en el capítulo 18, “Homonimia de operadores”).

Sugerencia de rendimiento 16.4

Pasar un objeto en llamada por valor es bueno desde un punto de vista de seguridad, porque la función llamada no tiene acceso al objeto original, pero cuando se tiene que hacer una copia de un objeto grande, la llamada por valor puede degradar el rendimiento. Un objeto puede ser pasado en llamada por referencia, pasando ya sea un apuntador o una referencia al objeto. La llamada por referencia ofrece un buen rendimiento, pero es más débil desde un punto de vista de seguridad, debido a que la función llamada tiene acceso al objeto original. Una alternativa segura sería una referencia en llamada por const.

16.17 Reutilización del software

Las personas que escriben programas orientados a objetos se concentran en la puesta en práctica de clases útiles. Existe una enorme oportunidad de capturar y catalogar clases, de tal forma que sean accesibles a grandes segmentos de la comunidad de programadores. Existen en todo el mundo muchas *bibliotecas de clases*, y otras están siendo desarrolladas. Se hacen esfuerzos para que dichas bibliotecas sean muy accesibles. El software podrá entonces ser construido a partir de componentes existentes, bien definidos, cuidadosamente probados, bien documentados, portátiles y ampliamente disponibles. Este tipo de *reutilización de software* puede acelerar el desarrollo de software poderoso y de alta calidad. El desarrollo rápido de aplicaciones (*RAD por Rapid applications development*) se convierte en posible.

Antes de que se pueda alcanzar el potencial total de la reutilización del software, deben ser resueltos, sin embargo, problemas significativos. Necesitamos procesos para catalogar, procedimientos para licenciar, mecanismos de protección para asegurar que las copias maestras de clases no están corrompidas, procedimientos de descripción, de tal forma que los diseñadores de sistemas nuevos puedan determinar si los objetos existentes llenan sus necesidades, mecanismos de búsqueda para determinar qué clases están disponibles y que tan justamente cumplen los requisitos del diseñador de software, y así en lo sucesivo. Muchos problemas interesantes de investigación y desarrollo necesitan resolverse. Estos problemas *serán* resueltos, porque es enorme el valor potencial de sus soluciones.

Resumen

- Las estructuras son tipos de datos agregados construidos utilizando objetos de otros tipos.
- La palabra reservada **struct** introduce la definición de estructura. El cuerpo de una estructura queda definido entre llaves ({ y }). La definición de cada estructura debe terminar con un punto y coma.
- Se puede utilizar un nombre de etiqueta de estructura para declarar variables de un tipo de estructura.
- Las definiciones de estructura no reservan espacio en memoria; crean nuevos tipos de datos que son utilizados para declarar variables.
- Se tiene acceso a los miembros de una estructura o de una clase utilizando los operadores de acceso de miembro el operador punto (.) y el operador flecha (->). El operador punto da acceso a un miembro de estructura vía el nombre de variable del objeto o una referencia al objeto. El operador flecha da acceso a un miembro de estructura vía un apuntador al objeto.

- Los inconvenientes para la creación de nuevos tipos de datos utilizando los **struct** de tipo C son la posibilidad de tener datos sin inicializar; inicializaciones incorrectas; todos los programas que utilicen **struct** del estilo C deben ser transformados si se modifica la puesta en práctica de **struct**; y no se da protección para asegurar que los datos se conservan en un estado consistente con los valores de datos apropiados.
- Las clases le permiten al programador hacer modelos de objetos con atributos y comportamientos. Los tipos de clase pueden ser definidos en C++ utilizando las palabras reservadas **class** y **struct**, pero la palabra reservada **class** por lo regular es utilizada para este fin.
- El nombre de clase puede ser utilizado para declarar objetos de dicha clase.
- Las definiciones de clase empiezan con la palabra reservada **class**. El cuerpo de la definición de clase está delimitado por llaves (**{** y **}**). Las definiciones de clase terminan con un punto y coma.
- Cualquier miembro de datos o función miembro declarada después de **public:** en una clase es visible a cualquier función que tenga acceso a un objeto de dicha clase.
- Cualquier miembro de dato o función miembro declarada después de **private:** es sólo visible a amigos y otros miembros de dicha clase.
- Los especificadores de acceso de miembro siempre terminan con dos puntos (**:**) y pueden aparecer múltiples veces en una definición de clase.
- Los datos privados no son accesibles desde fuera de la clase.
- La puesta en práctica de una clase se dice que está oculta de sus clientes, es decir, está "encapsulada".
- Un constructor es una función miembro especial, con el mismo nombre de la clase, y se utiliza para inicializar los miembros de un objeto de clase. Los constructores son llamados cuando se produce o se ejemplifica un objeto de su clase.
- La función con el mismo nombre que la clase, pero antecedita con el carácter tilde (~), se llama un destructor.
- El conjunto de funciones miembro públicas de una clase se conoce como la interfaz de la clase o la interfaz pública.
- Cuando una función miembro se define por fuera de la definición de la clase, el nombre de función es antecedido por el nombre de la clase y por el operador de resolución de alcance binario (**::**).
- Las funciones miembro definidas utilizando el operador de resolución de alcance por fuera de una definición de clase, están dentro del alcance de la clase.
- Las funciones miembro definidas en una definición de clase están en línea en forma automática. El compilador se reserva el derecho de no considerar en línea a cualquier función.
- Llamar funciones miembro es más conciso que la llamada de funciones en la programación procedural, porque las funciones miembro asumen que los nombres no declarados son miembros de la clase en la cual la función miembro está definida.
- Dentro del alcance de una clase, los miembros de la clase pueden ser referenciados simplemente con sus nombres. Fuera del alcance de una clase, los miembros de la clase se referencian ya sea a través de un nombre de objeto, una referencia a un objeto o un apuntador a un objeto.
- Los operadores de selección de miembros **.** y **->** son utilizados para tener acceso a los miembros de clase.

- Un principio fundamental de buena ingeniería de software es separar la interfaz de la puesta en práctica para facilitar la capacidad de modificación del programa.
- Las definiciones de clase por lo regular están colocadas en los archivos de cabecera y las definiciones de función miembro están normalmente colocadas en los archivos de código fuente, con el mismo nombre base.
- El modo de acceso por omisión para las clases es **private:** de tal forma que todos los miembros después de un encabezado de clase y antes de la primera etiqueta se consideran como privados.
- Los miembros de clase públicos presentan una vista de los servicios que proporciona la clase.
- El acceso a los datos privados de una clase puede ser controlado con cuidado mediante el uso de las funciones miembro conocidas como funciones de acceso. Si una clase desea permitir que los clientes lean datos privados, la clase puede proporcionar una función "get". Para permitir que los clientes modifiquen datos privados, la clase puede proveer una función "set".
- Los miembros de datos de una clase normalmente se hacen privados y las funciones miembro de una clase a menudo se hacen públicos. Algunas funciones miembro se conservan privadas y sirven como funciones de utilidad a otras funciones de la clase.
- Los miembros de datos de una clase no pueden ser inicializados en una definición de clase. Deben ser inicializados en un constructor, o sus valores pueden ser definidos más adelante, después de que su objeto haya sido creado.
- Los constructores pueden tener homónimos.
- Una vez que un objeto de clase esté correctamente inicializado, todas las funciones miembro que manipulan el objeto deberán asegurarse que el objeto se conserva en un estado consistente.
- Cuando se declara un objeto de una clase, pueden ser proporcionados los inicializadores. Estos inicializadores se pasan al constructor de la clase.
- Los constructores pueden especificar argumentos por omisión.
- Los constructores pudieran no especificar tipos de regreso, ni pudieran intentar regresar valores.
- Si no se define un constructor para una clase, el compilador crea un constructor por omisión. Un constructor por omisión suministrado por el compilador no lleva a cabo ninguna inicialización, por lo cual cuando se crea el objeto, no está garantizado que esté en un estado consistente.
- Un destructor de una clase es automáticamente llamado, cuando un objeto de una clase sale de alcance. El destructor mismo, de hecho no destruye el objeto, pero sí ejecuta trabajos de terminación, antes de que el sistema recupere el almacenamiento del objeto.
- Los destructores no reciben parámetros y no regresan valores. Una clase sólo puede tener un destructor.
- El operador de asignación (**=**) es utilizado para asignar un objeto a otro objeto del mismo tipo. Dicha asignación se ejecuta normalmente mediante copia a nivel de miembro —cada miembro de un objeto se copia de manera individual al mismo miembro en otro objeto.

Terminología

operador de dirección &
operador de referencia &

operador de resolución de alcance ::
tipo de datos abstractos (ADT)

función de acceso	interfaz a una clase
operador de asignación (=)	salir de alcance
atributo	control de acceso de miembro
comportamiento	especificador de acceso de miembro
class	función miembro
operador selector de miembro de clase (.)	inicializador de miembro
declaración de clase	copia a nivel de miembro
definición de clase	mensaje
alcance de clase	método
nombre de etiqueta de clase	función no miembro
cliente de una clase	objeto local no estático
estado consistente para un miembro de datos	objeto
constructor	programación orientada a objetos (OOP)
abstracción de datos	función predicada
ocultamiento de datos	principio del mínimo privilegio
miembro de datos	private:
tipo de datos	programación procedural
constructor por omisión	protected:
destructor	interfaz pública de una clase
objetos dinámicos	public:
encapsulado	función de consulta
alcance de entrada	desarrollo rápido de aplicaciones (RAD)
extensibilidad	código reutilizable
alcance de archivo	operador de resolución de alcance (::)
declaración hacia adelante de una clase	servicios de una clase
función get	función set
objeto global	reutilización de software
archivo de cabecera	archivo de código fuente
puesta en práctica de una clase	objeto local estático
ocultamiento de información	trabajos de terminación
inicializar un objeto de clase	tilde (~) en nombre de destructor
función miembro en línea	tipo definido por usuario
ejemplos de una clase	función de utilidad.
ejemplificación de un objeto de una clase	

Errores comunes de programación

- 16.1 Olvidar el punto y coma al final de una definición de clase.
- 16.2 Intentar inicializar explícitamente un miembro de datos de una clase.
- 16.3 Intentar hacer la homonimia de una función miembro mediante una función no incluida en el alcance de dicha clase.
- 16.4 Un intento por parte de una función, que no sea un miembro de una clase particular (o un amigo de dicha clase) de obtener acceso a un miembro privado de dicha clase.
- 16.5 Intentar inicializar de forma explícita un miembro de datos de una clase dentro de la definición de clase.
- 16.6 Intentar declarar un tipo de regreso para un constructor, y/o intentar regresar un valor proveniente de un constructor.
- 16.7 Intentar pasar argumentos a un destructor, regresar valores de un destructor, o hacer la homonimia de un destructor.
- 16.8 Un constructor puede llamar a otras funciones miembro de la clase como son las funciones set o get, pero dado que el constructor está inicializando el objeto, los miembros de datos pudieran no

estar aún en un estado consistente. Puede causar errores utilizar miembros de datos antes de que hayan sido adecuadamente inicializados.

Prácticas sanas de programación

- 16.1 Para mayor claridad y legibilidad, utilice cada especificador de acceso de miembro sólo una vez en una definición de clase. Coloque primero los miembros públicos, donde sean con facilidad localizables.
- 16.2 Defina todas, a excepción de las más pequeñas funciones miembro, por afuera de la definición de clase. Esto ayuda a separar la interfaz de la clase de su puesta en práctica.
- 16.3 Utilice en un programa las directivas de preprocesador **#ifndef**, **#define** y **#endif** para evitar que los archivos de cabecera sean incluidos más de una vez.
- 16.4 Utilice el nombre del archivo de cabecera, con el punto reemplazado por un subrayado, en las directivas de preprocesador **#ifndef** y **#define** correspondientes a un archivo de cabecera.
- 16.5 Si en una declaración de clase decide listar primero los miembros privados, utilice en forma explícita la etiqueta **private:** a pesar de que por omisión se supone que **private:** es asumido. Esto mejora la claridad del programa. Nuestra preferencia es listar primero los miembros **public:** de una clase.
- 16.6 A pesar del hecho que las etiquetas **public:** y **private:** pueden ser repetidas y entremezcladas, liste primero en un grupo todos los miembros públicos de una clase y a continuación liste en otro grupo todos los miembros privados. Esto enfoca la atención del cliente en la interfaz pública de la clase, en vez de en la puesta en práctica de la misma.
- 16.7 Evita confusión usar los especificadores de acceso de miembro **public:**, **protected:**, y **private:** una sola vez en cada definición de clase.
- 16.8 Cuando sea apropiado (casi siempre), proporcione un constructor para asegurarse que todo objeto es inicializado de manera correcta, con valores significativos.
- 16.9 Cada función miembro (y amigo) que modifique los miembros de datos privados de un objeto deberán asegurarse que los datos se conserven en un estado consistente.
- 16.10 Incluya siempre un constructor que ejecute la inicialización adecuada para su clase.
- 16.11 Las funciones miembro que definen los valores de los datos privados, deberán verificar que los nuevos valores propuestos son correctos; si no lo son, las funciones set deben colocar los miembros de datos privados en un estado consistente apropiado.
- 16.12 Nunca haga que una función miembro pública regrese una referencia no **const** (o un apuntador) a un miembro de datos privado. Regresar una referencia como ésta viola el encapsulado de la clase.

Sugerencias de rendimiento

- 16.1 Las estructuras por lo regular pasan en llamada por valor. Para evitar la sobrecarga de copiar una estructura, pase la estructura en llamada por referencia.
- 16.2 A fin de evitar la sobrecarga de la llamada por valor y aún así obtener el beneficio de que la información original del llamador quede protegida contra modificaciones, pase argumentos de tamaño extenso como referencias **const**.
- 16.3 Definir una función miembro pequeña dentro de una definición de clase automáticamente coloca la función miembro en línea (si el compilador así lo decide). Esto puede mejorar el rendimiento, pero no promueve una mejor ingeniería de software.
- 16.4 Pasar un objeto en llamada por valor es bueno desde un punto de vista de seguridad, porque la función llamada no tiene acceso al objeto original, pero cuando se tiene que hacer una copia de un objeto grande, la llamada por valor puede degradar el rendimiento. Un objeto puede ser pasado en llamada por referencia, pasando ya sea un apuntador o una referencia al objeto. La llamada por

referencia ofrece un buen rendimiento, pero es más débil desde un punto de vista de seguridad, debido a que la función llamada tiene acceso al objeto original. Una alternativa segura sería una referencia en llamada por `const`.

Observaciones de ingeniería de software

- 16.1 Es importante escribir programas que sean comprensibles y fáciles de mantener. Las modificaciones son la regla más que la excepción. Los programadores deberán prever que su código será modificado. Como veremos, las clases facilitan la capacidad de modificación de los programas.
- 16.2 Los clientes de una clase utilizan la clase sin conocer los detalles internos de cómo está puesta en práctica dicha clase. Si se modifica la puesta en práctica de la clase (para mejorar el rendimiento, por ejemplo) no es necesario modificar los clientes de la clase. Esto facilita mucho la modificación de sistemas.
- 16.3 Las funciones miembro por lo regular están formadas por unas pocas líneas de código, porque ninguna lógica es requerida para determinar si son válidos los miembros de datos.
- 16.4 Los clientes tienen acceso a la interfaz de una clase, pero no deberán tener acceso a la puesta en práctica de la clase.
- 16.5 Declarar funciones miembro dentro de una definición de clase y definir dichas funciones miembro por fuera de dicha definición de clase separa la interfaz de una clase de su puesta en práctica. Esto promueve buena ingeniería de software.
- 16.6 Usar un enfoque de programación orientado a objetos a menudo puede simplificar las llamadas de función al reducir el número de parámetros a pasarse. Este beneficio de la programación orientada a objetos se deriva del hecho que el encapsulado de los miembros de datos y las funciones miembros dentro de un objeto le da a las funciones miembro el derecho de acceso a los miembros de datos.
- 16.7 Coloque la declaración de la clase en un archivo de cabecera a incluirse por cualquier cliente que desee utilizar dicha clase. Esto forma la interfaz pública de la clase. Coloque las definiciones de las funciones miembro de la clase en un archivo fuente. Esto conforma la puesta en práctica de la clase.
- 16.8 Los clientes de una clase no necesitan ver el código fuente de la clase a fin de utilizarla. Los clientes deben, sin embargo, poder tener la capacidad de enlazarse con el código objeto de la clase.
- 16.9 Aquella información de importancia a la interfaz con una clase debería estar incluida en el archivo de cabecera. Aquella información que sólo será utilizada en forma interna en la clase y que no será necesaria para los clientes de la clase, debería ser incluida en el archivo fuente no publicado. Este es otra vez otro ejemplo del principio del mínimo privilegio.
- 16.10 C++ alienta a que los programas sean independientes de la puesta en práctica. Cuando se modifica la puesta en práctica de una clase utilizada mediante código independiente de la misma, dicho código no necesita ser cambiado, pero pudiera necesitar ser recompilado.
- 16.11 Conserve privados todos los miembros de datos de una clase. Proporcione funciones miembro públicas para definir los valores de los miembros de datos privados y para obtener los valores de los miembros de datos privados. Esta arquitectura ayuda a ocultar la puesta en práctica de una clase a la vista de sus clientes, lo que reduce errores y mejora la capacidad de modificación del programa.
- 16.12 Los diseñadores de clase utilizan miembros privados, protegidos y públicos para obligar a la idea de ocultamiento de información y al principio del mínimo privilegio.
- 16.13 Hacer privados los miembros de datos de una clase y públicas las funciones miembro de la clase, facilita la depuración, porque los problemas con las manipulaciones de datos se localizan ya sea en las funciones miembros de la clase o en los amigos de la misma.
- 16.14 Las funciones miembro tienen tendencia a agruparse en varias categorías diferentes: funciones que leen y regresan el valor de miembros de datos privados, funciones que definen el valor de miembros de datos privados, funciones que ponen en práctica las características de la clase, y funciones que ejecutan varias tareas mecánicas para la clase, como la inicialización de objetos de clase, la asignación de objetos de clase, la conversión entre clases y tipos incorporados o entre clases y otras clases, y el manejo de memoria para objetos de clase.

- 16.15 Hacer los miembros de datos privados y controlando su acceso, especialmente el acceso de escritura, a aquellos miembros de datos vía funciones miembro, ayuda a asegurar la integridad de los datos.
- 16.16 Tener acceso a datos privados mediante las funciones miembro `set` y `get` no solamente protege los miembros de datos contra la recepción de valores inválidos, sino también aísla a los clientes de la clase de la representación de los miembros de datos. Entonces, si por alguna razón cambia la representación de los datos (típicamente para reducir la cantidad de almacenamiento requerido o para mejorar el rendimiento), sólo las funciones miembro necesitan modificarse no es necesario que los clientes cambien, en tanto la interfaz proporcionada por las funciones miembro se conserve igual. Los clientes pueden, sin embargo, requerir ser recompilados.

Ejercicios de autoevaluación

- 16.1 Llene cada uno de los siguientes espacios en blanco:
 - a) La palabra reservada `struct` introduce una definición de estructura.
 - b) Se tiene acceso a los miembros de clase vía el operador `*` en conjunción con un objeto de clase o vía el operador `.*` en conjunción con un apuntador a un objeto de clase.
 - c) Los miembros de una clase especificados como `static` son sólo accesibles a las funciones miembro de la clase y amigos de la clase.
 - d) Un `static` es una función miembro especial utilizada para inicializar los miembros de datos de una clase.
 - e) El acceso por omisión para los miembros de una clase es `this`.
 - f) Una función `static` se utiliza para asignar valores a los miembros de datos privados de una clase.
 - g) `new` puede ser utilizada para asignar un objeto de una clase a otro objeto de la misma clase.
 - h) Las funciones miembro de una clase a menudo se hacen `static` y los miembros de datos de una clase se hacen por lo regular `static`.
 - i) Una función `static` se utiliza para recuperar valores de datos privados de una clase.
 - j) El conjunto de funciones miembro públicas de una clase se conoce como la `interfaz` de la clase.
 - k) La puesta en práctica de una clase se dice que está oculta de sus clientes o `private`.
 - l) Las palabras reservadas `public` y `private` pueden ser utilizadas para introducir una definición de clase.
 - m) Los miembros de una clase especificados como `static` son accesibles en cualquier parte en que un objeto de la clase esté en alcance.
- 16.2 Encuentre el o los errores en cada una de las siguientes y explique cómo corregirlo.
 - a) Suponga que se declara el siguiente prototipo en la clase `Time`

```
void ~Time(int);
```

- b) Lo siguiente es una definición parcial de la clase `Time`

```
class Time {
public:
    //function prototypes
private:
    int hour = 0;
    int minute = 0;
    int second = 0;
};
```

- c) Suponga el siguiente prototipo que se declara en la clase `Employee`

```
int Employee(const char *, const char *);
```

Respuestas a los ejercicios de autoevaluación

16.1 a) **struct**. b) punto (.), flecha (->). c) **private**. d) constructor. e) **private**. f) **set**. g) copia a nivel de miembro por omisión. h) **private**, **public**. i) **get**. j) interfaz. k) encapsulado. l). **class**, **struct** m) **public**.

- 16.2** a) Error: no se permite que los destructores regresen valores o tomen argumentos.
Corrección: elimine el tipo de regreso **void** y el parámetro **int** de la declaración.
- b) Error: los miembros no pueden ser inicializados de forma explícita en la definición de clase.
Corrección: elimine la inicialización explícita de la definición de clase e inicialice los miembros de datos en un constructor.
- c) Error: los constructores no pueden regresar valores.
Corrección: elimine el tipo de regreso **int** de la declaración.

Ejercicios

16.3 ¿Cuál es el propósito del operador de resolución de alcance?

16.4 Compare los conceptos de **struct** y de **class** en C++.

16.5 Proporcione un constructor que sea capaz de utilizar la hora actual de la función **time()** —declarada en el encabezado **time.h**— de la biblioteca estándar de C para inicializar un objeto de la clase **Time**.

16.6 Cree una clase llamada **Complex** para ejecutar aritmética con números complejos. Escriba un programa manejador para probar su clase.

Los números complejos tienen la forma

$$\text{realPart} + \text{imaginaryPart} * i$$

donde i es

$$\sqrt{-1}$$

Utilice variables de punto flotante para representar los datos privados de la clase. Proporcione una función constructor que permita que se inicialice un objeto de esta clase cuando sea declarado. El constructor deberá contener valores por omisión, por si no se proporcionan inicializadores. Proporcione funciones miembros públicas para cada uno de los siguientes:

- a) Suma de dos números **Complex**: las partes reales se suman juntas y las partes imaginarias se suman juntas también.
- b) Resta de dos números **Complex**: la parte real del operando derecho se resta de la parte real del operador izquierdo y la parte imaginaria del operando derecho se resta de la parte imaginaria del operador izquierdo.
- c) Imprimir números **Complex** en la forma (**a**, **b**) donde **a** sea la parte real y **b** la parte imaginaria.
- 16.7** Cree una clase llamada **RationalNumber** (para ejecutar aritmética con fracciones). Escriba un programa manejador para probar su clase.

Utilice variables enteras para representar los datos privados de la clase el numerador y el denominador. Proporcione una función constructor que permita que se inicialice un objeto de esta clase cuando sea declarado. El constructor deberá contener valores por omisión, para el caso en que no se proporcionen inicializadores y deberá almacenar la fracción en forma simplificada (es decir, la fracción

$$\frac{2}{4}$$

deberá ser almacenada en el objeto como 1 en el numerador y 2 en el denominador). Proporcione funciones miembro públicas para cada una de las siguientes:

- a) Suma de dos números **Rational**. El resultado deberá ser almacenado en forma simplificada.
- b) Resta de dos números **Rational**. El resultado deberá ser almacenado en forma simplificada.
- c) Multiplicación de dos números **Rational**. El resultado deberá ser almacenado en forma simplificada.
- d) División de dos números **Rational**. El resultado deberá ser almacenado en forma simplificada.
- e) Imprimir números **Rational** en la forma **a/b** donde **a** es el numerador y **b** el denominador.
- f) Imprimir números **Rational** en formato de punto flotante.

16.8 Modifique la clase **Time** de la figura 16.10 para incluir una función miembro **tick** que incremente la hora almacenado en un objeto **Time** en un segundo. El objeto **Time** deberá conservarse siempre en un estado consistente. Escriba un programa manejador que pruebe la función miembro **tick** en un ciclo que imprima la hora en formato estándar durante cada iteración del ciclo para ilustrar que la función miembro **tick** funciona correctamente. Asegúrese de probar los casos siguientes:

- a) Incrementar al siguiente minuto.
- b) Incrementar a la hora siguiente.
- c) Incrementar al día siguiente (es decir 11:59:59 PM a 12:00:00 AM).

16.9 Modifique la clase **Date** de la figura 16.12 para que lleve a cabo verificación de errores en los valores de inicializador correspondientes a los miembros de datos **month**, **day** y **year**. También proporcione una función miembro **nextDay** para incrementar los días en uno. El objeto **Date** deberá siempre conservarse en un estado consistente. Escriba un programa manejador que pruebe la función **nextDay** en un ciclo que imprima la fecha durante cada iteración del ciclo, a fin de ilustrar que la función **nextDay** funciona correctamente. Asegúrese de probar los casos siguientes:

- a) Incrementar al siguiente mes.
- b) Incrementar al siguiente año.

16.10 Combine la clase modificada **Time** del ejercicio 16.8 y la clase modificada **Date** del ejercicio 16.9 en una clase llamada **DateAndTime** (en el capítulo 19 analizaremos la herencia que nos permitirá llevar a cabo esta tarea rápidamente sin tener que modificar las definiciones existentes de clase). Modifique la función **tick** para llamar la función **nextDay** si la hora es incrementado al día siguiente. Modifique la función **printStandard** y **printMilitary** para extraer la fecha en adición a la hora. Escriba un programa manejador para probar la nueva clase **DateAndTime**. Pruebe específicamente incrementando la hora para pasar al día siguiente.

17

Clases: Parte II

Objetivos

- Ser capaz de crear y destruir dinámicamente objetos.
- Ser capaz de especificar objetos constantes y funciones miembro constantes.
- Comprender el objeto de las funciones amigo y de las clases amigo.
- Comprender el uso de los miembros de datos y de las funciones miembro estáticos.
- Comprender los varios tipos de clases contenedor.
- Ser capaz de desarrollar clases iteradoras que recorren los elementos de las clases contenedor.
- Comprender el uso del apuntador this.
- Ser capaz de crear y utilizar clases plantilla.

*¿Pero qué, para servir a nuestros propios intereses,
nos impide engañar a nuestros amigos?*

Charles Churchill

*En vez de esta absurda división de los sexos, deberían clasificar
a las personas como estáticas y dinámicas.*

Evelyn Waugh

Sobre todo lo siguiente: se veraz contigo mismo.

William Shakespeare

Hamlet

No tengas amigos que no sean tus iguales.

Confucius

Sinopsis

- 17.1 Introducción
- 17.2 Objetos constantes y funciones miembro `const`
- 17.3 Composición: clases como miembros de otras clases
- 17.4 Funciones amigo y clases amigo
- 17.5 Cómo utilizar el apuntador `this`
- 17.6 Asignación dinámica de memoria mediante los operadores `new` y `delete`
- 17.7 Miembros de clase estáticos
- 17.8 Abstracción de datos y ocultamiento de información
 - 17.8.1 Ejemplo: tipo de datos abstracto de arreglo
 - 17.8.2 Ejemplo: tipo de datos abstracto de cadena
 - 17.8.3 Ejemplo: tipo de datos abstracto de cola
- 17.9 Clases contenedor e iteradores
- 17.10 Clases plantilla

Resumen • Terminología • Errores comunes de programación • Prácticas sanas de programación • Sugerencias de rendimiento • Sugerencia de portabilidad • Observación de ingeniería de software • Ejercicios de autoevaluación • Respuestas a los ejercicios de autoevaluación • Ejercicios.

17.1 Introducción

En este capítulo continuamos nuestro estudio de las clases y de la abstracción de datos. Analizamos muchos temas más avanzados y colocamos los cimientos para el análisis de la homonimia de clases y de operadores del capítulo 18. El análisis de los capítulos 16 al 18 fomenta el uso de los objetos por parte de los programadores. Después, los capítulos 19 y 20 introducen herencia y polimorfismo las técnicas de la verdadera programación orientada a objetos.

17.2 Objetos constantes y funciones miembro `const`

Repetidamente hemos enfatizado el *principio del mínimo privilegio* como uno de los principios más fundamentales de una buena ingeniería de software. Veamos una forma en la cual este principio se aplica a los objetos.

Algunos objetos necesitan ser modificables y otros no. El programador puede utilizar la palabra reservada `const` para indicar que un objeto no es modificable, y que será un error cualquier intento de modificarlo. Por ejemplo,

```
const Time noon(12, 0, 0);
```

declara un objeto `const` de nombre `noon` de la clase `Time` y lo inicializa a las 12 de medio día.

Observación de ingeniería de software 17.1

Declarar un objeto como `const` ayuda a que se cumpla el principio del mínimo privilegio. Cualquier intento accidental de modificar dicho objeto será detectado en tiempo de compilación, en vez de causar errores en tiempo de ejecución.

Los compiladores C++ respetan de forma tan rígida las declaraciones `const`, que para objetos `const` no permiten de ninguna manera llamadas de función miembro (algunos compiladores sólo emiten una advertencia). Esto es duro, porque es probable que los clientes del objeto desearán utilizar varias funciones miembro “get” con dicho objeto. Para ello, el programador pudiera declarar funciones miembro `const`; sólo éstas pueden operar sobre objetos `const`. Naturalmente, las funciones miembro `const` no pueden modificar el objeto.

Error común de programación 17.1

Definir como `const` una función miembro que modifique un miembro de datos de un objeto.

Error común de programación 17.2

Definir como `const` una función miembro que llama a una función miembro no `const`.

Error común de programación 17.3

Llamar a una función miembro no `const` en relación con un objeto `const`.

Error común de programación 17.4

Intentar modificar un objeto `const`.

Una función es definida como `const` tanto en su declaración como en su definición, insertando la palabra reservada `const` después de la lista de parámetros de la función, y, en el caso de la definición de función, antes de la llave izquierda que inicia el cuerpo de la función. Por ejemplo, la función miembro

```
int getValue() const {return privateDataMember};
```

que solo regresa el valor de uno de los miembros de datos del objeto, es declarada como `const`. Si una función miembro `const` se define fuera de la definición de la clase, tanto la declaración como la definición de la función miembro deberán incluir `const`.

Observación de ingeniería de software 17.2

Una función miembro `const` puede ser homónima en una versión no `const`. El compilador selecciona de forma automática la función miembro homónima basándose en si el objeto ha sido declarado `const` o no.

Aquí se presenta un problema interesante tratándose de constructores y destructores, los cuales ciertamente modifican los objetos. Para constructores y destructores de objetos `const` no se requiere la declaración `const`. A un constructor debe permitírsele poder modificar un objeto de forma tal que el objeto pueda ser inicializado de manera correcta. Un destructor debe ser capaz de ejecutar sus trabajos de terminación, antes de que un objeto sea destruido.

El programa de la figura 17.1 ejemplifica un objeto constante de la clase `Time` e intenta modificar el objeto mediante las funciones miembro no constantes `setHour`, `setMinute` y `setSecond`. Las advertencias generadas por el compilador Borland C++ aparecen en la ventana de salida. El compilador fue ajustado de forma que no compile, si se produjese cualquier mensaje de advertencia.

Práctica sana de programación 17.1

Declare como `const` todas las funciones miembro que se pretenda utilizar con objetos `const`.

Un objeto `const` no puede ser modificado por asignación, por lo que deberá ser inicializado. Cuando un objeto miembro es `const`, un *inicializador de miembro* deberá ser utilizado para proporcionar al constructor con valores iniciales correspondientes al objeto. La figura 17.2 demuestra los usos de la sintaxis del inicializador de miembro para inicializar el miembro de datos `const` de nombre `increment` de la clase `Increment`. El constructor correspondiente a `Increment` se modifica como sigue:

```
// TIME5.H
// Declaration of the class Time.
// Member functions defined in TIME5.CPP

#ifndef TIME5_H
#define TIME5_H

class Time {
public:
    Time(int = 0, int = 0, int = 0); // default constructor

    // set functions
    void setTime(int, int, int); // set time
    void setHour(int);           // set hour
    void setMinute(int);         // set minute
    void setSecond(int);         // set second

    // get functions (normally declared const)
    int getHour() const;         // return hour
    int getMinute() const;       // return minute
    int getSecond() const;       // return second

    // print functions (normally declared const)
    void printMilitary() const;  // print military time
    void printStandard() const;  // print standard time
private:
    int hour;           // 0 - 23
    int minute;         // 0 - 59
    int second;         // 0 - 59
};

#endif
```

Fig. 17.1 Cómo utilizar una clase `Time` con objetos `const` y funciones miembro `const` (parte 1 de 4).

```
// TIME5.CPP
// Member function definitions for Time class.
#include "time5.h"
#include <iostream.h>

// Constructor function to initialize private data.
// Default values are 0 (see class definition).
Time::Time(int hr, int min, int sec)
{
    hour = (hr >= 0 && hr < 24) ? hr : 0;
    minute = (min >= 0 && min < 60) ? min : 0;
    second = (sec >= 0 && sec < 60) ? sec : 0;
}

// Set the values of hour, minute, and second.
void Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
}

// Set the hour value
void Time::setHour(int h)
{
    hour = (h >= 0 && h < 24) ? h : 0;
}

// Set the minute value
void Time::setMinute(int m)
{
    minute = (m >= 0 && m < 60) ? m : 0;
}

// Set the second value
void Time::setSecond(int s)
{
    second = (s >= 0 && s < 60) ? s : 0;
}

// Get the hour value
int Time::getHour() const { return hour; }

// Get the minute value
int Time::getMinute() const { return minute; }

// Get the second value
int Time::getSecond() const { return second; }

// Display military format time: HH:MM:SS
void Time::printMilitary() const
{
    cout << (hour < 10 ? "0" : "") << hour << ":"
        << (minute < 10 ? "0" : "") << minute << ":"
        << (second < 10 ? "0" : "") << second;
}
```

Fig. 17.1 Cómo utilizar una clase `Time` con objetos `const` y funciones miembro `const` (parte 2 de 4).

```
// Display standard format time: HH:MM:SS AM (or PM)
void Time::printStandard() const
{
    cout << ((hour == 12) ? 12 : hour % 12) << ":"
        << (minute < 10 ? "0" : "") << minute << ":"
        << (second < 10 ? "0" : "") << second
        << (hour < 12 ? " AM" : " PM");
}
```

Fig. 17.1 Cómo utilizar una clase **Time** con objetos **const** y funciones miembro **const** (parte 3 de 4).

```
// FIG17_1.CPP
// Attempting to access a const object with
// non-const member functions.
#include <iostream.h>
#include "time5.h"

main()
{
    const Time t(19, 33, 52);    // constant object

    t.setHour(12);    // ERROR: non-const member function
    t.setMinute(20);  // ERROR: non-const member function
    t.setSecond(39);  // ERROR: non-const member function

    return 0;
}
```

```
Compiling FIG17_1.CPP:
Warning FIG17_1.CPP: Non-const function
Time::setHour(int) called for const object
Warning FIG17_1.CPP: Non-const function
Time::setMinute(int) called for const object
Warning FIG17_1.CPP: Non-const function
Time::setSecond(int) called for const object
```

Fig. 17.1 Cómo utilizar una clase **Time** con objetos **const** y funciones miembro **const** (parte 4 de 4).

```
Increment::Increment(int c, int i)
: increment(i)
{ count = c; }
```

La notación : **increment(i)** hace que **increment** sea inicializado al valor **i**. Si se requieren de varios inicializadores de miembro, simplemente inclúyalos después de los dos puntos en una lista separada por comas.

```
// FIG17_2.CPP
// Using a member initializer to initialize a
// constant of a built-in data type.

#include <iostream.h>

class Increment {
public:
    Increment(int c = 0, int i = 1);
    void addIncrement() { count += increment; }
    void print() const;
private:
    int count;
    const int increment;    // const data member
};

// Constructor for class Increment
Increment::Increment(int c, int i)
: increment(i)    // Member initializer
{ count = c; }

// Print the data
void Increment::print() const
{
    cout << "count = " << count
        << ", increment = " << increment << endl;
}

main()
{
    Increment value(10, 5);

    cout << "Before incrementing: ";
    value.print();

    for (int j = 1; j <= 3; j++) {
        value.addIncrement();
        cout << "After increment " << j << ": ";
        value.print();
    }

    return 0;
}
```

```
Before incrementing: count = 10, increment = 5
After increment 1: count = 15, increment = 5
After increment 2: count = 20, increment = 5
After increment 3: count = 25, increment = 5
```

Fig. 17.2 Cómo utilizar un inicializador de miembro para inicializar una constante de un tipo de datos incorporado.

En la figura 17.3 se ilustran los errores de compilador para un programa que intenta inicializar **increment** mediante un enunciado de asignación, en vez de mediante un inicializador de miembro.

Error común de programación 17.5

No proporcionar un inicializador de miembro para un objeto miembro **const**.

Observación de ingeniería de software 17.3

Tanto los objetos **const** como las "variables" **const** necesitan ser inicializadas con sintaxis de inicializador miembro. Las asignaciones no son permitidas.

17.3 Composición: clases como miembros de otras clases

Un objeto de clase **AlarmClock** necesita saber cuándo se supone debe sonar su alarma, por lo que, ¿por qué no incluir un objeto **Time** como miembro del objeto **AlarmClock**? Tal capacidad se llama *composición*. Una clase puede tener otras clases como miembros.

Observación de ingeniería de software 17.4

Una forma de reutilización del software es la composición, en la cual una clase tiene como miembros objetos de otras clases.

Cuando un objeto entra en alcance, su constructor es llamado automáticamente, por lo que es preciso especificar cómo se pasan argumentos a constructores de objetos miembro. Se construyen los objetos miembro, antes de que los objetos de clase que los incluyen sean construidos.

Observación de ingeniería de software 17.5

Si un objeto tiene varios objetos miembro, está indefinido el orden en el cual los objetos miembro serán construidos. No escriba código que dependa de que los constructores de objetos miembro ejecuten su trabajo en un orden específico.

En la figura 17.4 la clase **Employee** y la clase **Date** se utilizan para demostrar objetos como miembros de otros objetos. La clase **Employee** contiene los miembros de datos privados **lastName**, **firstName**, **birthDate**, y **hireDate**. Los miembros **birthDate** y **hireDate** son objetos de la clase **Date** que contiene los miembros de datos privados **month**, **day** y **year**. El programa produce un objeto **Employee**, e inicializa y muestra sus miembros de datos. Note la sintaxis del encabezado de función en la definición de constructor **Employee**:

```
Employee::Employee(char *fname, char *lname,
                  int bmonth, int bday, int byear,
                  int hmonth, int hday, int hyear)
: birthDate(bmonth, bday, byear),
  hireDate(hmonth, hday, hyear)
```

El constructor toma ocho argumentos (**fname**, **lname**, **bmonth**, **bday**, **byear**, **hmonth**, **hday** y **hyear**). Los puntos en el encabezado separan los inicializadores de miembro de la lista de parámetros. Los inicializadores de miembro definen los argumentos **Employee** pasados a los constructores de los objetos miembro. Entonces, **bmonth**, **bday** y **byear** son pasados al constructor **birthDate**, y **hmonth**, **hday** y **hyear** son pasados al constructor **hireDate**. Varios inicializadores miembro son separados por comas.

```
// FIG17_3.CPP
// Attempting to initialize a constant of
// a built-in data type with an assignment.
#include <iostream.h>

class Increment {
public:
    Increment(int c = 0, int i = 1);
    void addIncrement() { count += increment; }
    void print() const;
private:
    int count;
    const int increment;
};

// Constructor for class Increment
Increment::Increment(int c, int i)
{
    // Constant member 'increment' is not initialized
    count = c;
    increment = i; // ERROR: Cannot modify a const object
}

// Print the data
void Increment::print() const
{
    cout << "count = " << count
          << ", increment = " << increment << '\n';
}

main()
{
    Increment value(10, 5);

    cout << "Before incrementing: ";
    value.print();

    for (int j = 1; j <= 3; j++) {
        value.addIncrement();
        cout << "After increment " << j << ": ";
        value.print();
    }

    return 0;
}
```

```
Compiling FIG17_3.CPP:
Warning FIG17_3.CPP 18: Constant member 'increment' is
not initialized
Error FIG17_3.CPP 20: Cannot modify a const object
Warning FIG17_3.CPP 21: Parameter 'i' is never used
```

Fig. 17.3 Intento erróneo de inicializar una constante de un tipo de datos incorporado mediante asignación.

```
// DATE1.H
// Declaration of the Date class.
// Member functions defined in DATE1.CPP
#ifndef DATE1_H
#define DATE1_H

class Date {
public:
    Date(int = 1, int = 1, int = 1900); // default constructor
    void print() const; // print date in month/day/year format

private:
    int month; // 1-12
    int day; // 1-31 based on month
    int year; // any year

    // utility function to test proper day for month and year
    int checkDay(int);
};

#endif
```

Fig. 17.4 Cómo utilizar los inicializadores objeto miembro (parte 1 de 5).

Un objeto miembro no necesita comenzar mediante un inicializador miembro. Si no se proporciona inicializador miembro, se llamará automáticamente al constructor por omisión del objeto miembro. Los valores, si es que hay alguno, que hayan sido definidos por el constructor por omisión, podrán ser pasados por alto mediante funciones set.

Error común de programación 17.6

No proporcionar un constructor por omisión para un objeto miembro, cuando no se proporciona inicializador miembro para dicho objeto miembro. Esto puede dar como resultado un objeto miembro no inicializado.

Sugerencia de rendimiento 17.1

Inicialice de forma explícita los objetos miembro mediante inicializadores miembro. Esto elimina la sobrecarga de una doble inicialización de objetos miembro una vez cuando se llame al constructor por omisión del objeto miembro, y una segunda vez cuando se utilicen las funciones set para inicializar dicho objeto miembro.

17.4 Funciones amigo y clases amigo

Una *función amigo* de una clase se define por fuera del alcance de dicha clase, pero aún así tiene el derecho de acceso a los miembros **private** (y como veremos en el capítulo 19, "Herencia", a los miembros **protected**) de la clase. Se puede declarar una función o toda una clase como un **friend** de otra clase.

Para declarar una función como un **friend** de una clase, en la definición de clase preceda el prototipo de función con la palabra reservada **friend**. Para declarar la clase **ClassTwo** como amigo de la clase **ClassOne**, prepare una declaración de la forma

```
friend ClassTwo;
```

como un miembro de la clase **ClassOne**.

```
// DATE1.CPP
// Member function definitions for Date class.

#include <iostream.h>
#include "date1.h"

// Constructor: Confirm proper value for month;
// call utility function checkDay to confirm proper
// value for day.
Date::Date(int mn, int dy, int yr)
{
    month = (mn > 0 && mn <= 12) ? mn : 1; // validate
    year = yr; // could also check
    day = checkDay(dy); // validate

    cout << "Date object constructor for date ";
    print();
    cout << endl;
}

// Utility function to confirm proper day value
// based on month and year.
int Date::checkDay(int testDay)
{
    static int daysPerMonth[13] = {0, 31, 28, 31, 30,
                                    31, 30, 31, 31, 30,
                                    31, 30, 31};

    if (month != 2) {
        if (testDay > 0 && testDay <= daysPerMonth[month])
            return testDay;
    }
    else { // February: Check for possible leap year
        int days = (year % 400 == 0 ||
                    (year % 4 == 0 && year % 100 != 0) ? 29 : 28);

        if (testDay > 0 && testDay <= days)
            return testDay;
    }

    cout << "Day " << testDay << " invalid. Set to day 1.\n";
    return 1; // leave object in consistent state if bad value
}

// Print Date object in form month/day/year
void Date::print() const
{ cout << month << '/' << day << '/' << year; }
```

Fig. 17.4 Cómo utilizar inicializadores objeto miembro (parte 2 de 5).

```
// EMPY1.H
// Declaration of the Employee class.
// Member functions defined in EMPY1.CPP
#ifndef EMPY1_H
#define EMPY1_H

#include "date1.h"

class Employee {
public:
    Employee(char *, char *, int, int, int, int, int, int);
    void print() const;
private:
    char lastName[25];
    char firstName[25];
    Date birthDate;
    Date hireDate;
};

#endif
```

Fig. 17.4 Cómo utilizar inicializadores objeto miembro (parte 3 de 5).

```
// EMPY1.CPP
// Member function definitions for Employee class.
#include <iostream.h>
#include <string.h>
#include "empty1.h"
#include "date1.h"

Employee::Employee(char *fname, char *lname,
                  int bmonth, int bday, int byear,
                  int hmonth, int hday, int hyear)
: birthDate(bmonth, bday, byear),
  hireDate(hmonth, hday, hyear)
{
    strncpy(firstName, fname, 24);
    firstName[24] = '\0';
    strncpy(lastName, lname, 24);
    lastName[24] = '\0';
    cout << "Employee object constructor: "
          << firstName << " " << lastName << endl;
}

void Employee::print() const
{
    cout << lastName << ", " << firstName << "\nHired: ";
    hireDate.print();
    cout << "  Birthday: ";
    birthDate.print();
    cout << endl;
}
```

Fig. 17.4 Cómo usar inicializadores objeto miembro (parte 4 de 5).

```
// FIG17_4.CPP
// Demonstrating an object with a member object.
#include <iostream.h>
#include "empty1.h"

main()
{
    Employee e("Bob", "Jones", 7, 24, 49, 3, 12, 88);

    cout << endl;
    e.print();

    cout << "\nTest Date constructor with invalid values:\n";
    Date d(14, 35, 94); // invalid Date values

    return 0;
}
```

```
Date object constructor for date 7/24/49
Date object constructor for date 3/12/88
Employee object constructor: Bob Jones

Jones, Bob
Hired: 3/12/88  Birthday: 7/24/49

Test Date constructor with invalid values:
Day 35 invalid. Set to day 1.
Date object constructor for date 1/1/94
```

Fig. 17.4 Cómo utilizar los inicializadores objeto miembro (parte 5 de 5).

Observación de ingeniería de software 17.6

Los conceptos de acceso de miembros correspondientes a **private**, **protected** y **public** no tienen relación con las declaraciones de amistad, por lo que las declaraciones de amistad pueden ser colocadas en cualquier parte de la definición de clase.

Práctica sana de programación 17.2

Coloque en la clase todas las definiciones de amistad en primer término, después del encabezado de clase, y no las anteceda con ningún especificador de acceso de miembros.

La amistad es concedida, y no tomada, es decir; para que la clase B sea un amigo de la clase A, la clase A debe declarar que la clase B es su amigo. También, la amistad no es ni simétrica ni transitiva, es decir, si la clase A es un amigo de la clase B, y la clase B es un amigo de la clase C, usted no puede inferir que la clase B sea un amigo de la clase A, que la clase C sea un amigo de la clase B o que la clase A sea un amigo de la clase C.

Observación de ingeniería de software 17.7

Algunas personas en la comunidad OOP (programación orientada a objetos) sienten que la "amistad" corrompe el ocultamiento de la información y debilita el valor del enfoque de diseño orientado a objetos.

El programa de la figura 17.5 demuestra la declaración y el uso de la función amigo `setX`, para definir el miembro de datos privado `x` de la clase `Count`. Note que en la declaración de clase la declaración `friend` aparece primero (por regla convencional), inclusive antes que las funciones miembro públicas sean declaradas. El programa de la figura 17.6 demuestra los mensajes producidos por el compilador cuando es llamada la función no amigo `cannotSetX`, para modificar el miembro de datos privado `x`.

Es posible especificar funciones homónimas como amigos de una clase. Cada función homónima que se desea como un amigo, debe ser declarada en forma explícita en la definición de clase como amigo de la clase.

```
// FIG17_5.CPP
// Friends can access private members of a class.
#include <iostream.h>

// Modified Count class
class Count {
    friend void setX(Count &, int); // friend declaration
public:
    Count() { x = 0; } // constructor
    void print() const { cout << x << endl; } // output
private:
    int x; // data member
};

// Can modify private data of Count because
// setX is declared as a friend function of Count
void setX(Count &c, int val)
{
    c.x = val; // legal: setX is a friend of Count
}

main()
{
    Count object;

    cout << "object.x after instantiation: ";
    object.print();
    cout << "object.x after call to setX friend function: ";
    setX(object, 8); // set x with a friend
    object.print();

    return 0;
}
```

```
object.x after instantiation: 0
object.x after call to setX friend function: 8
```

Fig. 17.5 Los amigos pueden tener acceso a los miembros privados de una clase.

```
// FIG17_6.CPP
// Non-friend/non-member functions cannot access
// private data of a class.
#include <iostream.h>

// Modified Count class
class Count {
public:
    Count() { x = 0; } // constructor
    void print() const { cout << x << '\n'; } // output
private:
    int x; // data member
};

// Function tries to modify private data of Count,
// but cannot because it is not a friend of Count.
void cannotSetX(Count &c, int val)
{
    c.x = val; // ERROR: 'Count::x' is not accessible
}

main()
{
    Count object;

    cannotSetX(object, 3); // cannotSetX is not a friend

    return 0;
}
```

```
Compiling FIG17_6.CPP:
Error FIG17_6.CPP 17: 'Count::x' is not accessible
Warning FIG17_6.CPP 18: Parameter 'c' is never used
Warning FIG17_6.CPP 18: Parameter 'val' is never used
```

Fig. 17.6 Funciones no amigas/no miembro no pueden tener acceso a los miembros privados de una clase.

17.5 Cómo utilizar el apuntador `this`

Cuando una función miembro referencia otro miembro de una clase en relación con un objeto específico de dicha clase, ¿Cómo C++ se asegura que es referenciado el objeto correcto? La respuesta es que cada objeto mantiene un apuntador a sí mismo llamado el *apuntador `this`* que es un argumento implícito en todas las referencias a miembros incluidos dentro de dicho objeto. El apuntador `this` puede también ser utilizado en forma explícita. Mediante el uso de la palabra reservada `this`, cada objeto puede determinar su propia dirección.

El apuntador `this` es utilizado de manera implícita para referenciar tanto los miembros de datos como las funciones miembro de un objeto. El tipo de este apuntador `this` depende del tipo del objeto y de si es declarada `const` la función miembro en la cual `this` es utilizado. En una función miembro no constante de la clase `Employee` el apuntador `this` tiene el tipo `Employee` * `const` (un apuntador constante a un objeto `Employee`). En una función miembro constante

de la clase **Employee** el apuntador **this** tiene el tipo **Employee const * const** (un apuntador constante a un objeto **Employee** constante).

Por ahora, presentamos un ejemplo sencillo de cómo utilizar de manera explícita el apuntador **this**; más adelante mostraremos algunos ejemplos substanciales y sutiles del uso de **this**. Cualquier función miembro tiene acceso al apuntador **this** al objeto para el cual está siendo invocado el miembro.

Sugerencia de rendimiento 17.2

Por razones de economía de almacenamiento, existe sólo una copia de cada función miembro por clase, y esta función miembro es invocada para todos los objetos de dicha clase. Por otra parte, cada objeto tiene su propia copia de los miembros de datos de la clase.

En la figura 17.7 se demuestra el uso explícito del apuntador **this** para permitir que una función miembro de la clase **Test** imprima los datos privados **x** de un objeto **Test**. El programa utiliza tanto el operador de flecha (**->**) sobre el apuntador **this**, como el operador de punto (**.**) sobre el apuntador **this** desreferenciado.

```
// FIG17_7.CPP
// Using the this pointer to refer to object members.
#include <iostream.h>

class Test {
public:
    Test(int = 0);
    void print() const;
private:
    int x;
};

Test::Test(int a) { x = a; } // constructor

void Test::print() const
{
    cout << "          x = " << x
          << "\n  this->x = " << this->x
          << "\n(*this).x = " << (*this).x << '\n';
}

main()
{
    Test a(12);

    a.print();

    return 0;
}
```

```
x = 12
this->x = 12
(*this).x = 12
```

Fig. 17.7 Cómo utilizar el apuntador **this**.

Note los paréntesis que encierran a ***this** cuando se utiliza con el operador de selección de miembro punto (**.**). Los paréntesis son necesarios, porque el operador punto tiene una precedencia más alta que el operador *****. Sin los paréntesis, la expresión

```
*this.x
```

sería evaluada como si tuviera los paréntesis como sigue:

```
*(this.x)
```

Esta expresión sería tratada por el compilador C++ como un error de sintaxis, porque el operador de selección de miembro no puede ser utilizado con un apuntador.

Error común de programación 17.7

*Intentar utilizar el operador de selección de miembro (**.**) con un apuntador a un objeto (el operador de selección de miembro sólo puede ser utilizado con un objeto o con una referencia a un objeto).*

Un uso interesante del apuntador **this**, es impedir que un objeto sea asignado a sí mismo. Como veremos en el capítulo 18, "Homonomia de operadores", la autoasignación puede causar errores serios cuando los objetos contengan apuntadores a almacenamientos asignados de forma dinámica mediante el operador **new**.

En la figura 17.8 se ilustra regresar una referencia a un objeto **Time** para permitir llamadas de función miembro de la clase **Time**, para concatenarlas. Cada una de las funciones miembro **Time** de nombre **setTime**, **setHour**, **setMinute** y **setSecond** regresan ***this** con un tipo de regreso **Time &**.

¿Porqué es válida la técnica de regresar ***this** como una referencia? El operador punto (**.**) se asocia de izquierda a derecha, por lo que la expresión

```
t.setHour(18).setMinute(30).setSecond(22);
```

primero evalúa **t.setHour(18)** y a continuación, regresa una referencia al objeto **t** como el valor de esta llamada de función. El resto de la expresión es entonces interpretado como

```
t.setMinute(30).setSecond(22);
```

La llamada **t.setMinute(30)** es ejecutada, regresando el equivalente de **t**. El resto de la expresión se interpreta como

```
t.setSecond(22);
```

Note que las llamadas

```
t.setTime(20, 20, 20).printStandard();
```

también utilizan la característica de concatenación. En esta expresión, estas llamadas deberán aparecer en este orden, porque **printStandard**, como está definida en la clase, no regresa una referencia a **t**. Colocar, en el enunciado anterior, la llamada a **printStandard** antes de la llamada a **setTime**, resultaría en un error de sintaxis.

```
// TIME6.H
// Declaration of class Time.
// Member functions defined in TIME6.CPP

#ifndef TIME6_H
#define TIME6_H

class Time {
public:
    Time(int = 0, int = 0, int = 0); // default constructor

    // set functions
    Time &setTime(int, int, int); // set hour, minute, second
    Time &setHour(int); // set hour
    Time &setMinute(int); // set minute
    Time &setSecond(int); // set second

    // get functions (normally declared const)
    int getHour() const; // return hour
    int getMinute() const; // return minute
    int getSecond() const; // return second

    // print functions (normally declared const)
    void printMilitary() const; // print military time
    void printStandard() const; // print standard time
private:
    int hour; // 0 - 23
    int minute; // 0 - 59
    int second; // 0 - 59
};

#endif
```

Fig. 17.8 Cómo encadenar llamadas de función miembro (parte 1 de 4).

```
// TIME6.CPP
// Member function definitions for Time class.

#include "time6.h"
#include <iostream.h>

// Constructor function to initialize private data.
// Default values are 0 (see class definition).
Time::Time(int hr, int min, int sec)
{
    hour = (hr >= 0 && hr < 24) ? hr : 0;
    minute = (min >= 0 && min < 60) ? min : 0;
    second = (sec >= 0 && sec < 60) ? sec : 0;
}
```

Fig. 17.8 Cómo encadenar llamadas de función miembro (parte 2 de 4).

```
// Set the values of hour, minute, and second.
Time &Time::setTime(int h, int m, int s)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    minute = (m >= 0 && m < 60) ? m : 0;
    second = (s >= 0 && s < 60) ? s : 0;
    return *this; // enables chaining
}

// Set the hour value
Time &Time::setHour(int h)
{
    hour = (h >= 0 && h < 24) ? h : 0;
    return *this; // enables chaining
}

// Set the minute value
Time &Time::setMinute(int m)
{
    minute = (m >= 0 && m < 60) ? m : 0;
    return *this; // enables chaining
}

// Set the second value
Time &Time::setSecond(int s)
{
    second = (s >= 0 && s < 60) ? s : 0;
    return *this; // enables chaining
}

// Get the hour value
int Time::getHour() const { return hour; }

// Get the minute value
int Time::getMinute() const { return minute; }

// Get the second value
int Time::getSecond() const { return second; }

// Display military format time: HH:MM:SS
void Time::printMilitary() const
{
    cout << (hour < 10 ? "0" : "") << hour << ":"
         << (minute < 10 ? "0" : "") << minute << ":"
         << (second < 10 ? "0" : "") << second;
}

// Display standard format time: HH:MM:SS AM (or PM)
void Time::printStandard() const
{
    cout << ((hour == 12) ? 12 : hour % 12) << ":"
         << (minute < 10 ? "0" : "") << minute << ":"
         << (second < 10 ? "0" : "") << second
         << (hour < 12 ? " AM" : " PM");
}
```

Fig. 17.8 Cómo encadenar llamadas de función miembro (parte 3 de 4).

```
// FIG17_8.CPP
// Chaining member function calls together
// with the this pointer
#include <iostream.h>
#include "time6.h"

main()
{
    Time t;

    t.setHour(18).setMinute(30).setSecond(22);
    cout << "Military time: ";
    t.printMilitary();
    cout << "\nStandard time: ";
    t.printStandard();

    cout << "\n\nNew standard time: ";
    t.setTime(20, 20, 20).printStandard();
    cout << endl;

    return 0;
}
```

```
Military time: 18:30:22
Standard time: 6:30:22 PM

New standard time: 8:20:20 PM
```

Fig. 17.8 Cómo encadenar llamadas de función miembro (parte 4 de 4).

17.6 Asignación dinámica de memoria mediante los operadores new y delete

Los operadores **new** y **delete** ofrecen una mejor forma de efectuar la asignación dinámica de memoria, que mediante las llamadas de función **malloc** y **free** de C. Considere el código siguiente

```
TypeName *typeNamePtr;
```

En ANSI C, para crear dinámicamente un objeto del tipo **TypeName**, usted diría

```
typeNamePtr = malloc(sizeof(TypeName));
```

Esto requiere una llamada de función a **malloc** y una referencia explícita al operador **sizeof**. En versiones de C anteriores a ANSI C, también tendría que haber hecho una conversión explícita (cast) del apuntador regresado por **malloc**, utilizando el cast (**TypeName ***). La función **malloc** no tiene ningún procedimiento para inicializar el bloque de memoria asignado. En C++, simplemente usted escribe

```
typeNamePtr = new TypeName;
```

El operador **new** crea en forma automática un objeto del tamaño apropiado, llama el constructor para el objeto (si existe uno disponible) y regresa un apuntador del tipo correcto. Si **new** no puede

encontrar espacio, regresa un apuntador 0. Para liberar espacio para este objeto en C++, utilice el operador **delete**, como sigue:

```
delete typeNamePtr;
```

C++ le permite incluir un *inicializador* para un objeto recién creado como en:

```
float *thingPtr = new float (3.14159);
```

el cual inicializa a 3.14159 un objeto **float** recién creado.

Se puede crear un arreglo y asignarlo a **int * chessBoardPtr** como sigue:

```
chessBoardPtr = new int[8][8];
```

Este arreglo puede ser borrado mediante el enunciado

```
delete [] chessBoardPtr;
```

Como veremos, utilizar **new** y **delete**, en vez de **malloc** y **free**, también ofrece otros beneficios. En particular, **new** de manera automática invoca al constructor, y **delete** automáticamente invoca al destructor de la clase.

Error común de programación 17.8

*Mezclar asignación dinámica de memoria del tipo **new** y **delete** con asignación dinámica de memoria del tipo **malloc** y **free**: el espacio creado por **malloc** no podrá ser liberado por **delete**; los objetos creados mediante **new** no podrán ser borrados por **free**.*

Práctica sana de programación 17.3

*A pesar de que los programas C++ pueden contener almacenamiento creado por **malloc** y borrados por **free**, y objetos creados por **new** y borrados por **delete**, lo mejor es utilizar solo **new** y **delete**.*

17.7 Miembros de clase estáticos

Por lo regular, cada objeto de una clase tiene su propia copia de todos los miembros de datos de la clase. Esto a veces resulta un desperdicio. En ciertos casos todos los miembros de una clase deberían compartir una copia de un miembro de datos particular. Para esta y otras razones se utiliza un miembro de datos estático. Un miembro de datos estático representa información "aplicable a toda la clase". La declaración de un miembro estático empieza con la palabra reservada **static**.

Sugerencia de rendimiento 17.3

Cuando una copia de los datos sea suficiente, utilice miembros de datos estáticos a fin de ahorrar almacenamiento.

Aunque los miembros de datos estáticos pudieran parecer variables globales, los miembros de datos estáticos tienen alcance de clase. Los miembros estáticos pueden ser públicos, privados o protegidos. Los miembros de datos estáticos deben ser inicializados en alcance de archivo. Los miembros de clase estáticos públicos son accesibles a través de cualquier objeto de dicha clase o mediante el operador de resolución de alcance binario, se puede tener acceso a ellos a través del nombre de la clase. Para tener acceso a los miembros de clase estáticos privados y protegidos se deben utilizar las funciones miembro públicas de la clase. Los miembros de clase estáticos existen aun cuando no existan objetos de dicha clase. Para tener acceso a un miembro de clase estático público cuando no existan objetos de dicha clase, sólo coloque el nombre de clase y el operador de resolución de alcance binario como prefijo al miembro de datos. Para tener acceso a un miembro

de clase estático privado o protegido, cuando no existan objetos de dicha clase, deberá ser incluida una función miembro estática pública y la función deberá ser llamada mediante el uso de un prefijo en su nombre con el nombre de la clase junto con el operador de resolución de alcance binario.

El programa de la figura 17.9 demuestra el uso del miembro de datos `static` privado y de la función miembro `static` pública. El miembro de datos `count` es inicializado a cero en alcance de archivo, mediante el enunciado

```
int Employee::count = 0;
```

El miembro de datos `count` lleva cuenta del número de objetos de la clase `Employee` que han sido producidos. Cuando existan objetos de la clase `Employee`, el miembro `count` puede ser referenciado a través de cualquier función miembro de un objeto `Employee` —en este ejemplo, `count` es referenciado tanto por el constructor como por el destructor. Cuando no existan objetos de la clase `Employee`, aun así el miembro `count` se puede referenciar, pero sólo mediante una llamada a la función miembro estática `getCount`, como sigue:

```
Employee::getCount()
```

En este ejemplo, se utiliza la función `getCount` para determinar el número de objetos `Employee` en la actualidad producidos. Note que cuando no exista ningún objeto producido, la llamada de función ya citada será emitida. Sin embargo, se podrá llamar a la función `getCount` a través de uno de los objeto cuando existan objetos producidos, como en

```
e1Ptr->getCount()
```

```
// EMPLOY1.H
// An employee class
#ifdef EMPLOY1_H
#define EMPLOY1_H

class Employee {
public:
    Employee(const char*, const char*); // constructor
    ~Employee();                       // destructor
    char *getFirstName() const;        // return first name
    char *getLastName() const;        // return last name

    // static member function
    static int getCount(); // return # objects instantiated
private:
    char *firstName;
    char *lastName;

    // static data member
    static int count; // number of objects instantiated
};

#endif
```

Fig. 17.9 Cómo utilizar un miembro de datos estático para llevar cuenta del número de objetos de una clase (parte 1 de 5).

```
// EMPLOY1.CPP
// Member functions definitions for class Employee
#include <iostream.h>
#include <string.h>
#include <assert.h>
#include "employ1.h"

// Initialize the static data member
int Employee::count = 0;

// Define the static member function that
// returns the number of employee objects instantiated.
int Employee::getCount() { return count; }

// Constructor dynamically allocates space for the
// first and last name and uses strcpy to copy
// the first and last names into the object
Employee::Employee(const char *first, const char *last)
{
    firstName = new char[ strlen(first) + 1 ];
    assert(firstName != 0); // ensure memory allocated
    strcpy(firstName, first);

    lastName = new char[ strlen(last) + 1 ];
    assert(lastName != 0); // ensure memory allocated
    strcpy(lastName, last);

    ++count; // increment static count of employees
    cout << "Employee constructor for " << firstName
         << ' ' << lastName << " called.\n";
}

// Destructor deallocates dynamically allocated memory
Employee::~~Employee()
{
    cout << "~Employee() called for " << firstName
         << ' ' << lastName << endl;
    delete [] firstName; // recapture memory
    delete [] lastName; // recapture memory
    --count; // decrement static count of employees
}

// Return first name of employee
char *Employee::getFirstName() const
{
    char *tempPtr = new char[strlen(firstName) + 1];
    assert(tempPtr != 0); // ensure memory allocated
    strcpy(tempPtr, firstName);
    return tempPtr;
}
```

Fig. 17.9 Cómo utilizar un miembro de datos estático para llevar cuenta del número de objetos de una clase (parte 2 de 5).

```
// Return last name of employee
char *Employee::getLastName() const
{
    char *tempPtr = new char[strlen(lastName) + 1];
    assert(tempPtr != 0); // ensure memory allocated
    strcpy(tempPtr, lastName);
    return tempPtr;
}
```

Fig. 17.9 Cómo utilizar un miembro de datos estático para llevar cuenta del número de objetos de una clase (parte 3 de 5).

```
// FIG17_9.CPP
// Driver to test the employee class
#include <iostream.h>
#include "employ1.h"

main()
{
    cout << "Number of employees before instantiation is "
         << Employee::getCount() << endl; // use class name

    Employee *e1Ptr = new Employee("Susan", "Baker");
    Employee *e2Ptr = new Employee("Robert", "Jones");

    cout << "Number of employees after instantiation is "
         << e1Ptr->getCount() << endl;

    cout << "\nEmployee 1: "
         << e1Ptr->getFirstName()
         << " " << e1Ptr->getLastName()
         << "\nEmployee 2: "
         << e2Ptr->getFirstName()
         << " " << e2Ptr->getLastName() << "\n\n";

    delete e1Ptr; // recapture memory
    delete e2Ptr; // recapture memory

    cout << "Number of employees after deletion is "
         << Employee::getCount() << endl;

    return 0;
}
```

Fig. 17.9 Cómo utilizar un miembro de datos estático para llevar cuenta del número de objetos de una clase (parte 4 de 5).

Una función miembro puede ser declarada **static** si no tiene acceso a miembros de clase no estáticos. A diferencia de las funciones miembro no estáticas, una función miembro estática no tiene un apuntador **this**, porque los miembros de datos estáticos y las funciones miembro estáticos existen en forma independiente a cualquier objeto de la clase.

```
Number of employees before instantiation is 0
Employee constructor for Susan Baker called.
Employee constructor for Robert Jones called.
Number of employees after instantiation is 2

Employee 1: Susan Baker
Employee 2: Robert Jones

~Employee() calle for Susan Baker
~Employee() calle for Robert Jones
Number of employees after deletion is 0
```

Fig. 17.9 Cómo utilizar un miembro de datos estático para llevar cuenta del número de objetos de una clase (parte 5 de 5).

Error común de programación 17.9

Referirse al apuntador **this** desde dentro de una función miembro estática.

Error común de programación 17.10

Declarar una función miembro estática **const**.

Observación de ingeniería de software 17.8

Los miembros de datos estáticos y las funciones miembro estáticas existen y pueden ser utilizados, aún si no se han producido objetos de dicha clase.

17.8 Abstracción de datos y ocultamiento de información

Las clases ocultan por lo regular sus detalles de puesta en práctica a los clientes (es decir, usuarios) de las clases. Esto se conoce como ocultamiento de la información. Por consiguiente, por ejemplo, el programador puede crear un clase de pila y ocultar a sus usuarios la puesta en práctica de la pila. Mediante arreglos o listas enlazadas las pilas pueden ser puestas en práctica en forma fácil. Un cliente de una clase de pila no necesita saber cómo ha sido puesta en práctica la misma. Lo que en realidad interesa al usuario es que cuándo en la pila se coloquen elementos de datos que éstos serán recuperados en orden de últimas entradas, primeras salidas (LIFO). Este concepto se conoce como *abstracción de datos*, y las clases de C++ definen *tipos de datos abstractos* (ADT, por *abstract data types*). Aunque los usuarios pudieran conocer los detalles de cómo se pone en práctica una clase, los usuarios no deben escribir código que dependa de dichos detalles. Esto significa que una clase particular (como una que ponga en práctica una pila, junto con sus operaciones **push** y **pop**), puede ser reemplazada por otra versión, sin afectar al resto del sistema, siempre y cuando la interfaz pública de dicha clase no cambie.

La tarea de un lenguaje de alto nivel es crear una presentación conveniente para uso de los programadores. No existe una forma aceptada de presentación y esta es una razón por la cual existen tantos lenguajes de programación. La programación orientada a objetos de C++, presenta sólo otra alternativa.

La mayor parte de los lenguajes de programación enfatizan acciones. En estos lenguajes, los datos existen en apoyo de acciones que los programas necesitan llevar a cabo. De todas formas, los datos son "menos interesantes" que las acciones. Los datos son "corrientes". Sólo existen unos

cuantos tipos de datos incorporados, y para los programadores resulta difícil crear sus propios tipos de datos.

En C++ y en el estilo de programación orientado a objetos, esta panorámica cambia. C++ aumenta la importancia de los datos. En C++ la actividad primordial es la creación de nuevos tipos de datos (es decir clases) y expresar las interacciones entre objetos de dichos tipos de datos.

Para ir en esa dirección, la comunidad de los lenguajes de programación necesitaba formalizar algunos conceptos relacionados con los datos. La formalización a la que nos estamos refiriendo es el concepto de tipo de datos abstracto (ADT por abstract data type). ADT recibe tanta atención, hoy en día, como tuvo durante las dos últimas décadas la programación estructurada. ADT no sustituye la programación estructurada. Más bien, provee una formalización adicional, que puede mejorar aún más el proceso de desarrollo de programas.

¿Qué es un tipo de datos abstracto? Veamos el tipo incorporado `int`. Lo que viene a la mente es el concepto de un entero en matemáticas, pero en una computadora `int` no es precisamente lo que es un entero en matemáticas. En particular, por lo regular los `int` de computadora son muy limitados en tamaño. Por ejemplo, en una máquina de 32 bits `int` pudiera estar limitado al rango desde -2 mil millones hasta +2 mil millones. Si el resultado de un cálculo cae fuera de este rango, ocurrirá un error y la máquina responderá de alguna manera dependiente de la máquina. Los enteros matemáticos no tienen este problema. Por lo tanto, el concepto de un `int` de computadora es solamente una aproximación al concepto de un entero del mundo real. Lo mismo es cierto tratándose de `float`.

Inclusive `char` es también una aproximación. A menudo los valores del tipo `char` son patrones de 8 bits, que no se parecen en nada a los caracteres que se supone deben representar como una **Z**, una **z**, un signo de dólares (\$), un dígito (por ejemplo, 5), y así en lo sucesivo. En la mayor parte de las computadoras los valores del tipo `char` están bastante limitados, en comparación con el rango de los caracteres del mundo real. El conjunto de caracteres ASCII de 7 bits proporciona 127 distintos valores de caracteres. Esto resulta en totalidad inadecuado para representar lenguajes como el japonés y el chino, que requieren de miles de caracteres.

El punto es que inclusive los tipos de datos incorporados, proporcionados en lenguajes de programación como C++, son sólo aproximaciones o modelos de conceptos y comportamientos del mundo real. Hasta este momento hemos aceptado como es a `int`, pero ahora el lector tiene que pensar con una nueva perspectiva. Los tipos como `int`, `float`, `char` y otros son todos ejemplos de *tipos de datos abstractos*. Son en esencia formas de representar conceptos del mundo real, a cierto nivel satisfactorio de precisión, dentro de un sistema de computación.

Un tipo de datos abstracto de hecho captura dos conceptos, es decir una representación de datos, así como aquellas operaciones permitidas sobre dichos datos. Por ejemplo, en C++, el concepto de `int` define la adición, sustracción, multiplicación, división y operaciones de módulo, pero queda indefinida la división entre cero; y estas operaciones permitidas se llevan a cabo de una forma que resulta dependiente de los parámetros de la máquina, como son el tamaño de la palabra fija del sistema de computación subyacente. En C++, para poner en práctica tipos de datos abstractos, el programador utiliza las clases.

17.8.1 Ejemplo: tipo de datos abstracto de arreglo

En el capítulo 6, analizamos los arreglos. Un arreglo no es mucho más que un apuntador. Si el programador es cuidadoso, esta capacidad primitiva es aceptable para efectuar operaciones de arreglos. Existen muchas operaciones que sería bueno ejecutar con arreglos, pero que en C++ no están incorporadas. Con C++, el programador puede desarrollar un arreglo ADT, que es preferible a los arreglos "en bruto". La clase arreglo puede proveer muchas nuevas capacidades como

- Verificación del rango de los subíndices.
- Un rango arbitrario de subíndices.
- Asignación de arreglo.
- Comparación de arreglo.
- Entrada/salida de arreglo.
- Los arreglos conocen su tamaño.

La debilidad aquí es que estamos creando un tipo de datos personalizado, no estándar, que es poco probable que esté disponible precisamente de esa forma, en la mayor parte de las puestas en práctica de C++. El uso de la programación orientada a objetos y de C++ está aumentando con rapidez. Resulta crucial que la profesión de la programación propugne por una normalización y distribución a gran escala de bibliotecas de clases, para poder aprovechar todo el potencial de la orientación a objetos.

C++ tiene un pequeño conjunto de tipos incorporados. ADT amplía el lenguaje de programación base.

Observación de ingeniería de software 17.9

El programador puede crear nuevos tipos mediante el uso del mecanismo de clases. Estos nuevos tipos pueden ser diseñados para ser usados tan convenientemente como los tipos incorporados. Por lo tanto, C++ resulta un lenguaje extensible. A pesar de que mediante estos nuevos tipos el lenguaje es fácil de ampliar, el lenguaje base mismo no es modificable.

Los nuevos ADT creados en entornos C++ pueden ser propiedad de un individuo, de pequeños grupos o de empresas. Los ADT también pueden ser colocados en bibliotecas de clases estándar destinadas a una amplia difusión. Esto no necesariamente promueve la normalización, aunque de hecho es probable que emergerán estándares. El valor pleno de C++ únicamente será comprendido cuando bibliotecas de clase sustanciales y estándar resulten ampliamente disponibles. Un proceso institucional debe ser instaurado para alentar el desarrollo de bibliotecas estándar. En los Estados Unidos, a menudo dicha normalización ocurre a través de ANSI, el American National Standards Institute. ANSI está desarrollando actualmente una versión estándar de C++. Independiente de cómo aparezcan estas bibliotecas en última instancia, el lector que aprenda C++ y programación orientada a objetos, estará listo para rápidamente aprovechar los nuevos tipos de desarrollo de software orientado a componentes, que serán posibles con las bibliotecas de ADT.

17.8.2 Ejemplo: tipo de datos abstracto de cadena

Intencionalmente C++ es un lenguaje escueto, que proporciona a los programadores sólo aquellas capacidades fundamentales necesarias para elaborar un amplio rango de sistemas. Los diseñadores del lenguaje no deseaban crear sobrecargas de rendimiento. Su objetivo era que C++ fuese apropiado para la programación de sistemas, que demandan que los programas se ejecuten con eficiencia. Es cierto, entre los tipos incorporados de C++, hubiera sido posible incluir un tipo de cadena, pero en vez de ello los diseñadores optaron incluir un mecanismo mediante clases para la creación y puesta en práctica de dichos ADT. En el capítulo 18 desarrollaremos nuestro propio ADT para cadenas.

17.8.3 Ejemplo: tipo de datos abstracto de cola

Cada uno de nosotros de vez en cuando tiene que esperar en fila. Una fila de espera también se llama una *cola*. Esperamos en fila en la caja de salida de un supermercado, esperamos en fila para

obtener gasolina, esperamos en fila para subimos al autobús, esperamos en fila en la autopista para pagar el peaje, y los estudiantes saben demasiado bien sobre tener que esperar en fila durante la época de registro para obtener las materias que desean. Los sistemas de cómputo utilizan de forma interna muchas filas de espera, y necesitamos escribir programas que simulen lo que las colas hacen y son.

Una cola es un bonito ejemplo de un tipo de datos abstracto. Una cola ofrece a sus clientes un comportamiento bien comprendido. Los clientes ponen cosas en una cola uno a la vez utilizando una operación *enqueue*, y los clientes obtienen estas cosas de regreso sobre demanda uno a la vez utilizando la operación *dequeue*. Conceptualmente, una cola puede hacerse muy larga. Una cola real, por lo regular, es finita. Los elementos de una cola son devueltos en un orden de *primeras entradas, primeras salidas (FIFO por first-in, first-out)*.

La cola oculta una representación de datos internos, que de alguna forma lleva control de los elementos actualmente esperando en fila, y ofrece un conjunto de operaciones a sus clientes, es decir *enqueue* y *dequeue*. Los clientes no tienen que preocuparse respecto a la puesta en práctica de la cola. Los clientes solo desean que la cola funcione "como anunciado". Cuando un cliente pone un nuevo elemento en cola, ésta deberá aceptar dicho elemento y colocarlo internamente en algún tipo de estructura de primeras entradas, primeras salidas. Cuando el cliente desee el siguiente elemento de la parte frontal de la cola, la cola deberá quitar el elemento de su representación interna y entregar el elemento al mundo exterior en un orden fijo, es decir, el elemento que ha estado por más tiempo en la cola, deberá ser el siguiente que, en la siguiente operación *dequeue*, será devuelto.

El ADT de la cola garantiza la integridad de su estructura interna de datos. Los clientes no pueden manipular en directo esta estructura de datos. Solo el ADT de la cola tiene acceso a sus datos internos. Los clientes pueden causar sólo operaciones permitidas, para ejecutarse sobre la representación de los datos; las operaciones no contempladas en la interfaz pública del ADT son rechazadas por el ADT de alguna forma apropiada. Esto podría significar la emisión de un mensaje de error, la terminación de la ejecución o sólo ignorar la solicitud de operación.

17.9 Clases contenedor e iteradores

Entre los tipos más populares de clases son las *clases contenedor* (también llamadas *clases de colección*), es decir, clases diseñadas para contener colecciones de objetos. Las clases contenedor proporcionan por lo común servicios como es inserción, borrado, búsqueda, clasificación, prueba de un elemento verificando membresía dentro de la clase, y otras similares. Los arreglos y las listas enlazadas son ejemplos de clases contenedor.

Es común asociar *objetos de iterador* o solo *iteradores* con la clase de colección. Un iterador es un objeto que regresa el siguiente elemento de una colección. Una vez que se ha escrito un iterador para una clase, obtener el siguiente elemento de la clase puede ser expresado en forma simple. Los iteradores se escriben como amigos de las clases, a través de las cuales hacen la iteración. De la misma forma que un libro se comparte entre varias personas y podría tener a la vez varios marcatextos, una clase contenedor puede tener varios iteradores operando sobre ella simultáneamente.

17.10 Clases plantilla

En el capítulo 15 analizamos funciones plantilla y cómo ayudan a la reutilización del software. Cerramos este capítulo con un análisis de las *clases plantilla*.

Es posible comprender que una pila es algo independiente del tipo de elementos que se colocan dentro de la pila. Pero cuando se trata de hecho programar una pila, se debe proporcionar un tipo

de datos. Esto crea una maravillosa oportunidad para la reutilización del software. Lo que necesitamos es un medio de describir el concepto genérico de pila y de producir clases que sean copias de esta clase genérica, pero de tipo específico. Esta es la capacidad proporcionada en C++ por las clases plantilla.

Observación de ingeniería de software 17.10

Las clases plantilla fomentan la reutilización del software.

Las clases plantilla se pusieron a disposición con el compilador de C++ de AT&T Versión 3, por lo que son una adición relativa reciente al lenguaje.

Las clases plantilla a menudo se conocen como *tipos parametrizados*, porque requieren de uno o más parámetros de tipo para especificar cómo personalizar la especificación de plantilla genérica.

El programador que desee utilizar clases plantilla, sólo escribe una definición de clase plantilla genérica. Cada vez que el programador necesite una nueva clase, el programador utiliza una notación concisa y simple, y el compilador escribe el código fuente para la clase que especifica el programador. Una clase plantilla de pila, por ejemplo podría entonces convertirse en base para crear muchas clases de pilas (como sería "una pila de *float*", "una pila de *int*", "una pila de *char*", etcétera) utilizadas en un programa.

Note la definición de la clase plantilla de pila en la figura 17.10. Se parece a una definición de clase convencional, excepto que está precedida por el encabezado

```
template <class T>
```

para indicar que se trata de una definición de clase plantilla que recibe un parámetro de tipo *T* indicando el tipo de la clase de pila que el programador desea crear. El tipo de elemento a almacenarse en esta pila se menciona sólo genéricamente como *T* a todo lo largo del encabezado de clase de pila y de las definiciones de función miembro.

```
// TSTACK1.H
// Simple template class Stack

#ifndef TSTACK1_H
#define TSTACK1_H

template <class T>
class Stack {
public:
    Stack(int = 10);           // Constructor with default size 10
    ~Stack() { delete [] stackPtr; } // Destructor
    int push(const T&);        // Push an element onto the stack
    int pop(T&);               // Pop an element off the stack
    int isEmpty() const { return top == -1; } // 1 if empty
    int isFull() const { return top == size - 1; } // 1 if full
private:
    int size;                 // # of elements in the stack
    int top;                  // location of the top element
    T *stackPtr;              // pointer to the stack
};
```

Fig. 17.10 Definición de la clase plantilla *Stack* (parte 1 de 4).

```
// Constructor with default size 10
template <class T>
Stack<T>::Stack(int s)
{
    size = s;
    top = -1; // Empty stack
    stackPtr = new T[size];
}

// Push an element onto the stack
// return 1 if successful, 0 otherwise
template <class T>
int Stack<T>::push(const T &item)
{
    if (!isFull()) {
        stackPtr[++top] = item;
        return 1; // push successful
    }
    return 0; // push unsuccessful
}

// Pop an element off the stack
template <class T>
int Stack<T>::pop(T &popValue)
{
    if (!isEmpty()) {
        popValue = stackPtr[top--];
        return 1; // pop successful
    }
    return 0; // pop unsuccessful
}

#endif
```

Fig. 17.10 Definición de la clase plantilla **Stack** (parte 2 de 4).

Ahora veamos el manejador de prueba (la función **main**) correspondiente a la clase plantilla de pila. El manejador empieza produciendo el objeto **floatStack** del tamaño 5. Este objeto se declara ser de la clase **Stack<float>**. El compilador asocia el tipo **float** con el tipo parametrizado **T** en la plantilla para producir el código fuente para una clase de pila del tipo **float**.

El manejador a continuación inserta (push) los valores **float** 1.1, 2.2, 3.3, 4.4 y 5.5 sobre **floatStack**. El ciclo de inserción termina cuando el manejador intenta insertar un sexto valor sobre **floatStack** (mismo que ya está lleno porque fue creado con 5 elementos).

Ahora el manejador extrae (pop) los 5 valores de la pila (en orden LIFO). El manejador intenta extraer un sexto valor, pero **floatStack** está vacío, por lo que el ciclo de extracción termina.

A continuación, el manejador produce una pila de enteros con la declaración

```
Stack<int>    intStack;
```

Dado que no se especifica tamaño, el tamaño por omisión será de 10, tal y como se especifica en el constructor por omisión. Otra vez, el manejador cicla la inserción de valores sobre **intStack**, hasta que éste queda lleno, y a continuación cicla la extracción de valores de **intStack** hasta que queda vacío.

```
// FIG17_10.CPP
// Test driver for Stack template

#include <iostream.h>
#include "tstack1.h"

main()
{
    Stack<float> floatStack(5);
    float f = 1.1;
    cout << "Pushing elements onto floatStack\n";

    while (floatStack.push(f)) { // success (1 returned)
        cout << f << ' ';
        f += 1.1;
    }

    cout << "\nStack is full. Cannot push " << f
        << "\n\nPopping elements from floatStack" << endl;

    while (floatStack.pop(f)) // success (1 returned)
        cout << f << ' ';

    cout << "\nStack is empty. Cannot pop" << endl;

    Stack<int> intStack;
    int i = 1;
    cout << "\nPushing elements onto intStack\n";

    while (intStack.push(i)) { // success (1 returned)
        cout << i << ' ';
        i++;
    }

    cout << "\nStack is full. Cannot push " << i
        << "\n\nPopping elements from intStack" << endl;

    while (intStack.pop(i)) // success (1 returned)
        cout << i << ' ';

    cout << "\nStack is empty. Cannot pop" << endl;
    return 0;
}
```

Fig. 17.10 Manejador para la clase plantilla **Stack** (parte 3 de 4).

Las definiciones de función miembro por fuera del encabezado de clase plantilla cada una de ellas empieza con el encabezado

```
template <class T>
```

Entonces cada definición se parece a una definición convencional, a excepción que el tipo de elemento de pila siempre se lista genéricamente como del tipo parametrizado **T**. Como siempre,

```

Pushing elements onto floatStack
1.1 2.2 3.3 4.4 5.5
Stack is full. Cannot push 6.6

Popping elements from floatStack
5.5 4.4 3.3 2.2 1.1
Stack is empty. Cannot pop

Pushing elements onto intStack
1 2 3 4 5 6 7 8 9 10
Stack is full. Cannot push 11

Popping elements from intStack
10 9 8 7 6 5 4 3 2 1
Stack is empty. Cannot pop

```

Fig. 17.10 Manejador para la clase plantilla `Stack` (parte 4 de 4).

se utiliza el operador de resolución de alcance binario para unir cada definición de función miembro al alcance correcto de la clase. En este caso, el nombre de clase es `Stack<T>`. Cuando `floatStack` es producido para ser del tipo `Stack<float>`, para representar la pila el constructor `Stack` crea un arreglo de elementos del tipo `T`. El tipo `float` sustituye a `T` y el enunciado

```
stackPtr = new T[size];
```

es generado por el compilador en la versión `Stackat<float>`, como

```
stackPtr = new float[size];
```

Los tipos parametrizados (es decir, las plantillas) pueden tomar más de un parámetro, como en

```
template <class T, class S, class R
```

Cada parámetro debe estar precedido por la palabra reservada `class`.

Resumen

- La palabra reservada `const` puede ser utilizada para especificar que un objeto no es modificable y que cualquier intento de modificarlo es un error.
- El compilador de C++ no permite llamadas de función miembro no `const` para un objeto `const`.
- Las funciones miembro `const` no pueden modificar un objeto.
- Una función se especifica como `const` tanto en su declaración como en su definición.
- Una función miembro `const` puede ser homónima con una versión no `const`. El compilador escogerá qué función miembro homónima utilizar en forma automática, basado en si el objeto ha sido declarado `const` o no.
- Un objeto `const` debe ser inicializado.

- Cuando una clase contenga objetos miembro `const`, los inicializadores miembros deben ser proporcionados al constructor.
- Las clases pueden estar compuestas de objetos de otras clases.
- Los objetos miembros son construidos antes de que sean construidos sus objetos de clase que los encierran.
- Si un inicializador miembro no es proporcionado para un objeto miembro, se llamará al constructor por omisión del objeto miembro.
- Una función amigo de una clase es una función definida por fuera de dicha clase y tiene derecho de acceso a miembros `private` y `protected` de la clase.
- Las declaraciones de amistad pueden ser colocadas en cualquier parte de la definición de clase.
- Cada función homónima que se pretende sea un amigo debe ser declarada de forma explícita como amigo de la clase.
- Cada objeto mantiene un apuntador a sí mismo, llamado apuntador `this`, que se convierte en un argumento implícito en todas las referencias a miembros dentro de dicho objeto. Este apuntador `this` es utilizado de forma implícita para referenciar tanto las funciones miembro como los miembros de datos de un objeto.
- Cada objeto puede determinar su propia dirección utilizando la palabra reservada `this`.
- El apuntador `this` puede ser utilizado explícitamente, pero se utiliza de manera implícita más a menudo.
- El operador `new` crea en forma automática un objeto del tamaño correcto, y regresa un apuntador del tipo correcto. En C++, para liberar el espacio correspondiente a este objeto, usted utiliza el operador `delete`.
- Un miembro de datos estático representa información "accesible a toda la clase". La declaración de un miembro estático empieza con la palabra reservada `static`.
- Los miembros de datos estáticos tienen alcance de clase.
- Los miembros estáticos de una clase son accesibles a través de un objeto de dicha clase, o pueden ser accesibles a través del nombre de la clase mediante el uso del operador de resolución de alcance.
- Una función miembro puede ser declarada estática, si no tiene acceso a miembros de clase no estáticos. A diferencia de las funciones miembro no estáticas, una función miembro estática no tiene apuntador `this`. Esto es debido a que los miembros de datos estáticos y las funciones miembro estáticas existen en forma independiente de cualquiera de los objetos de una clase.
- Las clases plantilla proporcionan una forma genérica de describir el concepto de una clase y de producir clases que son copias de esta clase genérica, pero son de tipo específico.
- Las clases plantilla a menudo se llaman tipos parametrizados, porque requieren de uno o más parámetros de tipo para especificar cómo personalizar la especificación de plantilla genérica.
- Las definiciones de clases plantilla empiezan con la línea

```
template <class T>
```

en la línea que antecede a la definición de clase. Puede existir más de un tipo parametrizado. Si es así, estarán separados por comas y cada tipo estará precedido por la palabra reservada `class`.